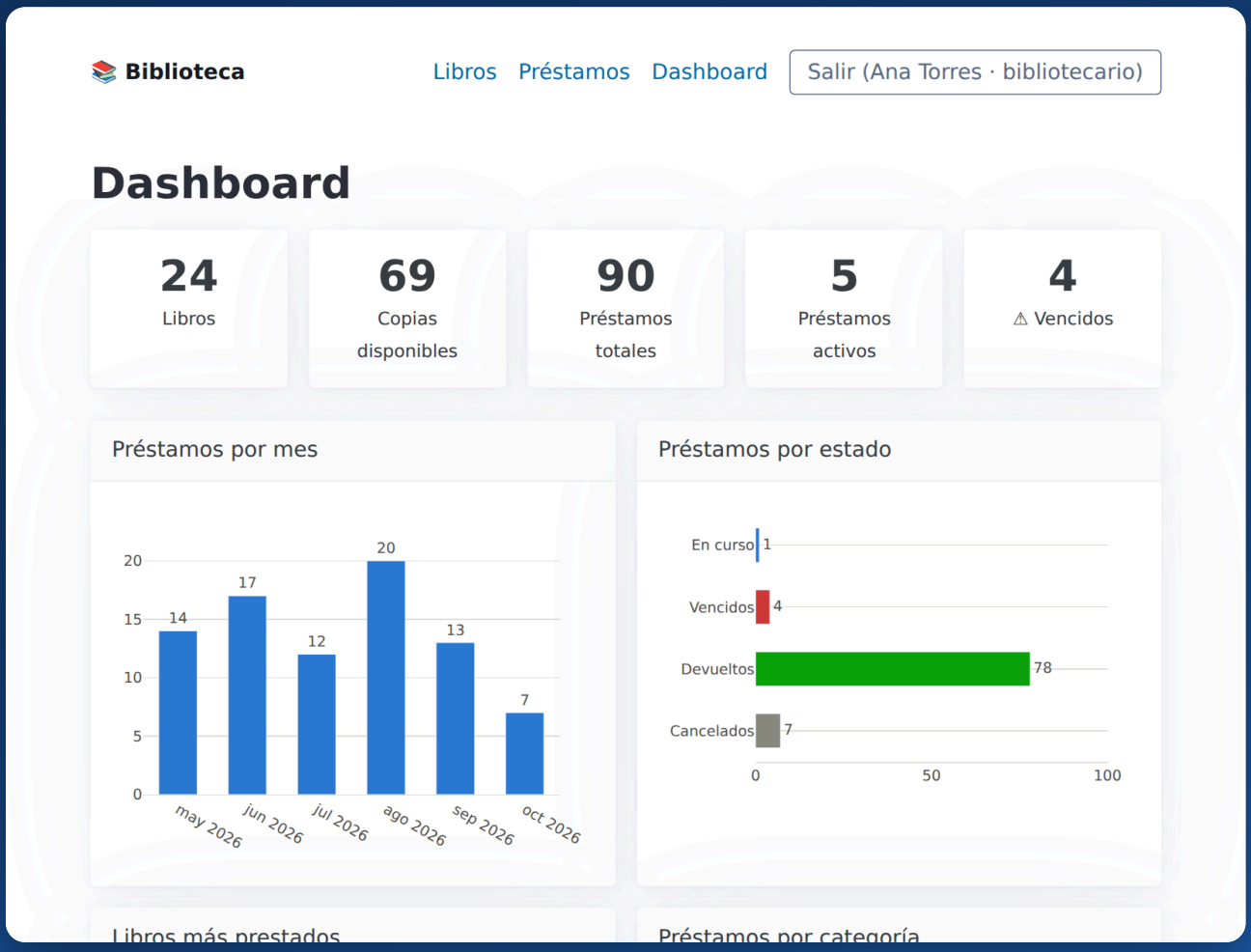
GUÍA DEL ESTUDIANTE · 4 CLASES

Biblioteca con Django y Firebase

Guía paso a paso para construir una aplicación fullstack 100 % en Python

Django 6.1 | Firebase Auth | Firestore | Plotly | Responsive



Octubre 2026 · Todo el código de esta guía fue verificado paso a paso

Contenido

Antes de empezar **3**

Qué vas a construir 3

Requisitos 5

Cómo leer esta guía 6

El plan de las 4 clases 7

Reglas de oro 7

Cómo está organizado el proyecto terminado 7

Clase 1 · Cimientos: Firebase, proyecto y autenticación **9**

1.1 Qué vamos a hacer hoy 9

1.2 Conceptos clave (sin miedo) 9

1.3 Crear el proyecto en Firebase 10

1.4 Preparar la carpeta y el entorno virtual 11

1.5 Instalar las dependencias 12

1.6 Crear el proyecto Django 13

1.7 Secretos: .env, la llave y .gitignore 14

1.8 Organizar el proyecto por features 15

1.9 Configurar settings.py 16

1.10 Las rutas del proyecto 19

1.11 core: lo que comparten todas las features 19

1.12 accounts: login, registro y logout 23

1.13 Crear tus usuarios con un comando 28

1.14 Probar en el navegador 29

1.15 Tests de la clase 1 31

1.16 Errores comunes de la clase 1 34

Clase 2 · Catálogo de libros, roles y datos de prueba **35**

2.1 Qué vamos a hacer hoy 35

2.2 Roles: quién puede hacer qué 35

2.3 El modelo de datos de un libro 36

2.4 Paginación reutilizable en core 36

2.5 La feature books 37

2.6 Conectar books con el proyecto 43

2.7 Probar el catálogo 44

2.8 El seeder: datos de prueba 45

2.9 Tests de la clase 2 50

2.10 Errores comunes de la clase 2 54

Clase 3 · Préstamos: prestar, devolver y cancelar **55**

3.1 El flujo de un préstamo 55
3.2 La feature loans 56
3.3 rules.py: las reglas como funciones puras 56
3.4 services.py: transacciones en Firestore 59
3.5 Vistas y rutas 61
3.6 Plantillas de loans 63
3.7 Conectar loans con el proyecto 64
3.8 Probar los préstamos 66
3.9 Tests de las vistas 68
3.10 Errores comunes de la clase 3 71

Clase 4 · Dashboard, diseño responsive y cierre 72

4.1 Qué vamos a hacer hoy 72
4.2 stats.py: de datos a números 72
4.3 charts.py: gráficas con Plotly 74
4.4 Vista, ruta y plantilla del dashboard 76
4.5 Probar el dashboard 78
4.6 Diseño responsive 79
4.7 Tests de la clase 4 88
4.8 Errores comunes de la clase 4 91
4.9 ¡Terminaste! 91

Apéndices 93

A · Comandos de referencia 93
B · Glosario 93
C · Lista de entrega 94

Antes de empezar

Qué vas a construir

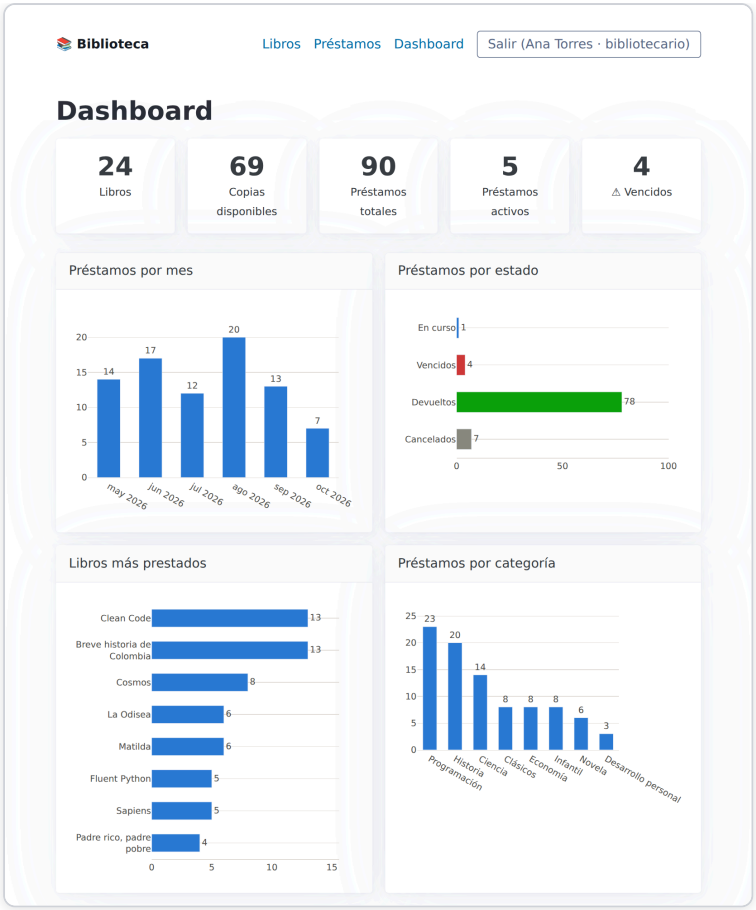
Una aplicación web de **biblioteca**, escrita 100 % en Python: el *frontend* (lo que ve el usuario) y el *backend* (la lógica y los datos). Al terminar tendrás:

- **Inicio de sesión** con correo y contraseña, y dos roles: *estudiante* y *bibliotecario*.
- **Catálogo de libros** con búsqueda, filtro por categoría y paginación. El bibliotecario crea, edita y elimina libros.
- **Préstamos:** el estudiante presta, devuelve y cancela. El sistema aplica reglas (máximo 3 préstamos activos, sin copias repetidas, plazo de 14 días) y protege el stock con transacciones.
- **Dashboard** con indicadores y cuatro gráficas sobre los préstamos.
- Un **seeder** que crea 24 libros y 90 préstamos de prueba.
- Un diseño **responsive** (celular, tablet y computador) y **71 pruebas automáticas**.



Todo el código (frontend y backend) es Python. Los usuarios nunca hablan directo con Firebase: lo hace el servidor.

Figura: la aplicación completa



El dashboard del bibliotecario, el resultado final de la clase 4



El catálogo de libros en el computador (clase 4)

> **Las capturas usan datos ficticios**
>
> Los nombres y correos que ves en las capturas (Ana Torres, Carlos Ruiz…) son inventados. Tú verás los usuarios que crees.

Requisitos

Necesitas	Detalle
Python 3.12 o superior	Comprueba con `python3 --version` (en Windows: `python --version`)
Un editor de código	Recomendado: Visual Studio Code
Un navegador moderno	Chrome, Edge o Firefox actualizados
Una cuenta de Google	Para crear el proyecto en Firebase (gratis)
Conexión a internet	El estilo (Pico CSS) y las gráficas (Plotly) se cargan desde internet
Conocimientos previos	Python básico: variables, listas, diccionarios, funciones y clases simples. **No** necesitas saber Django ni HTML avanzado

Necesitas	Detalle
Git (opcional)	Solo si quieres versionar tu proyecto

Cómo leer esta guía

Cada paso muestra **qué archivo** tocar y **qué escribir**. Estas etiquetas te indican qué hacer con el código:

Etiqueta	Significado
ARCHIVO NUEVO	Crea el archivo (y las carpetas que falten) y pega **todo** el contenido
REEMPLAZA TODO EL ARCHIVO	El archivo ya existe: borra su contenido y pega el nuevo
MODIFICA	Edita un archivo existente: busca el bloque **BUSCA** y cámbialo por el bloque **REEMPLAZA POR**
AGREGA AL FINAL	Pega el código al **final** del archivo, después de lo que ya tiene
TERMINAL	Comandos que escribes en la terminal, dentro de la carpeta `biblioteca`

Además encontrarás recuadros de color:

> **Nota**
>
> Información extra para entender mejor.

> **Consejo**
>
> Un atajo o una buena práctica.

> **Cuidado**
>
> Algo que puede salirte mal si no lo lees.

> **Peligro**
>
> Algo que no debes hacer nunca (por ejemplo, subir tus claves a internet).

> **✓ Verificación**
>
> Cómo comprobar que el paso salió bien **antes** de seguir.

El plan de las 4 clases

Clase	Tema	Lo que funciona al terminar
1	Cimientos: Firebase, proyecto y autenticación	Registro, login, logout y roles
2	Catálogo de libros, roles y datos de prueba	CRUD de libros, búsqueda, paginación y seeder
3	Préstamos	Prestar, devolver y cancelar con reglas y transacciones
4	Dashboard, responsive y cierre	Gráficas, diseño adaptable y 71 pruebas en verde

Cada clase está pensada para unas **3 horas** y termina con un punto de verificación. Si no lo terminas, retómala en tu casa: el proyecto siempre queda funcionando al final de cada clase.

Reglas de oro

1. **Copia exacto.** Python y los archivos de plantilla distinguen mayúsculas, espacios y sangrías. Usa **4 espacios** para sangrar (en VS Code, la tecla Tab ya lo hace).
2. **Guarda cada archivo** (Ctrl + S) antes de probar. El servidor se recarga solo cuando guardas.
3. **No avances si una verificación (✔) falla.** Un error de la clase 1 aparecerá disfrazado en la clase 3.
4. **Lee el error completo.** Casi siempre la **última línea** del mensaje te dice qué pasó, y la penúltima, en qué archivo.
5. **Activa el entorno virtual** cada vez que abras una terminal nueva. Si ves `(.venv)` al inicio de la línea, está activo.
6. **Nunca subas a internet** tu archivo `.env` ni la carpeta `firebase/`.

Cómo está organizado el proyecto terminado

Para que sepas hacia dónde vamos, este es el mapa del proyecto completo. No necesitas memorizarlo: lo iremos construyendo carpeta por carpeta.

ESTRUCTURA

```
biblioteca/
├── .venv/                          ← entorno virtual (lo crea python -m venv)
├── config/                         ← configuración global del proyecto
│   ├── asgi.py
│   ├── settings.py                 ← configuración principal
│   ├── urls.py                     ← rutas del proyecto
│   └── wsgi.py
├── core/                           ← lo compartido por todas las features
│   ├── static/                     ← estilos (site.css)
│   │   └── core/
│   │       └── css/
│   │           └── site.css
│   ├── templates/                  ← base.html y fragmentos compartidos
│   │   └── core/
│   │       ├── _form.html
│   │       ├── _pagination.html
```

```
│   │       └── base.html
│   ├── context_processors.py
│   ├── decorators.py                       ← permisos por rol
│   ├── firebase.py                         ← conexión única a Firebase
│   ├── pagination.py                       ← paginación reutilizable
│   ├── testing.py
│   ├── tests.py
│   ├── urls.py
│   └── views.py
├── features/                               ← una carpeta por funcionalidad
│   ├── accounts/                           ← login, registro y logout
│   │   ├── management/
│   │   │   └── commands/
│   │   │       └── create_user.py
│   │   ├── templates/
│   │   │   └── accounts/
│   │   │       ├── login.html
│   │   │       └── register.html
│   │   ├── forms.py
│   │   ├── services.py
│   │   ├── tests.py
│   │   ├── urls.py
│   │   └── views.py
│   ├── books/                              ← catálogo de libros
│   │   ├── templates/
│   │   │   └── books/
│   │   │       ├── form.html
│   │   │       └── list.html
│   │   ├── forms.py
│   │   ├── services.py
│   │   ├── tests.py
│   │   ├── urls.py
│   │   └── views.py
│   ├── dashboard/                          ← estadísticas y gráficas
│   │   ├── templates/
│   │   │   └── dashboard/
│   │   │       └── index.html
│   │   ├── charts.py
│   │   ├── stats.py
│   │   ├── tests.py
│   │   ├── urls.py
│   │   └── views.py
│   ├── loans/                              ← préstamos
│   │   ├── templates/
│   │   │   └── loans/
│   │   │       ├── _table.html
│   │   │       ├── all.html
│   │   │       └── mine.html
│   │   ├── rules.py                        ← reglas de negocio (funciones puras)
│   │   ├── services.py
│   │   ├── tests.py
│   │   ├── urls.py
│   │   └── views.py
│   └── seeder/                             ← datos de prueba
│       ├── management/
│       │   └── commands/
│       │       └── seed.py
│       └── tests.py
├── firebase/                               ← la llave privada de Firebase
│   └── serviceAccountKey.json
├── .env                                    ← secretos (no se sube a git)
├── .gitignore                              ← archivos que git ignora
├── db.sqlite3                              ← sesiones de Django (lo crea migrate)
├── manage.py                               ← comando para controlar el proyecto
└── requirements.txt                        ← librerías con su versión
```

CLASE 1

Cimientos: Firebase, proyecto y autenticación

Duración estimada: 3 horas · **Al terminar:** podrás registrarte, iniciar sesión y cerrar sesión, y tu aplicación sabrá si eres *bibliotecario* o *estudiante*.

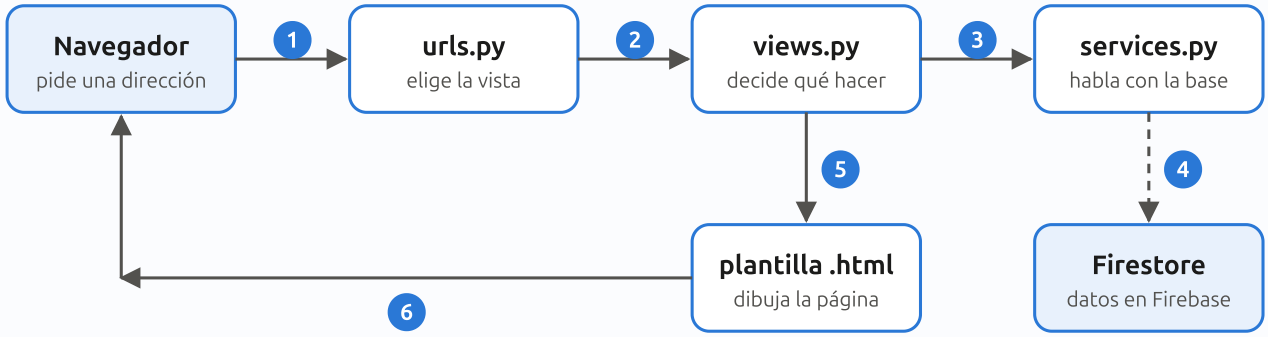
1.1 Qué vamos a hacer hoy

1. Crear un proyecto en **Firebase** (la base de datos y el sistema de usuarios).
2. Crear el proyecto **Django** desde cero, con su entorno virtual.
3. Conectar Django con Firebase de forma segura (sin escribir claves en el código).
4. Organizar el proyecto por *features* (explicado en la sección 1.8).
5. Construir el login, el registro y el logout.
6. Crear tus primeros usuarios y probar todo en el navegador.

> **Cómo trabajar esta guía**
>
> Sigue los pasos **en orden** y no te saltes las verificaciones (✔). Cada clase termina con `python manage.py test` : si todo sale en verde, puedes continuar con tranquilidad. Si algo falla, busca el mensaje de error en la tabla de errores comunes al final de la clase.

1.2 Conceptos clave (sin miedo)

Django es un framework web de Python. Cuando alguien abre una dirección en el navegador, Django sigue siempre el mismo camino:



1 petición · 2 la vista · 3 el servicio · 4 lee o escribe · 5 se arma el HTML · 6 respuesta al navegador

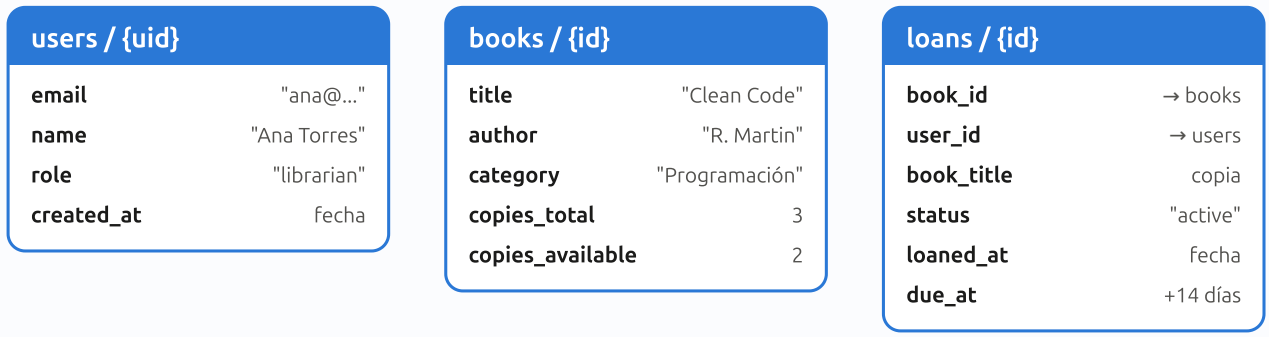
Figura: el camino de una petición en nuestro proyecto

- La **URL** decide qué vista atiende la petición (`urls.py`).
- La **vista** es una función de Python que decide qué hacer (`views.py`).
- El **servicio** habla con la base de datos (`services.py`). Esta capa la agregamos nosotros para que las vistas queden limpias.
- La **plantilla** es el HTML que se le devuelve al usuario (`templates/`).

Firebase es una plataforma de Google. Usaremos dos de sus servicios:

Servicio	Para qué lo usamos	Equivalente que ya conoces
Authentication	Guardar usuarios y verificar correo y contraseña	La tabla de usuarios de cualquier sitio web
Firestore	Guardar libros, préstamos y perfiles	Una base de datos, pero de *documentos* en lugar de tablas

Firestore organiza la información en **colecciones** (como carpetas) que contienen **documentos** (como fichas). Cada documento es parecido a un diccionario de Python:



Cada caja es una colección y muestra un documento de ejemplo.
Los campos con → apuntan al documento de otra colección.

Figura: las tres colecciones de Firestore

Nuestro servidor Python se conecta a Firebase de dos maneras:

- **Admin SDK** (librería `firebase-admin`): acceso total a Firestore y a la lista de usuarios, usando una *llave privada*. Solo vive en el servidor.
- **API REST** (librería `requests`): la única forma de comprobar una contraseña. Le enviamos correo y contraseña a Google y nos responde si son correctos.

1.3 Crear el proyecto en Firebase

Este paso se hace **en el navegador**, una sola vez. Los nombres de los menús pueden aparecer en español o en inglés; te damos ambos.

1. Entra a https://console.firebase.google.com con tu cuenta de Google.
2. Pulsa **Crear un proyecto** (*Create a project*). Ponle un nombre, por ejemplo `biblioteca-clase` . Cuando pregunte por Google Analytics, **desactívalo**: no lo necesitamos. Pulsa **Crear proyecto**.

3. En el menú de la izquierda abre **Compilación → Firestore Database** (*Build → Firestore Database*) y pulsa **Crear base de datos**. Elige la ubicación más cercana a ti y el modo **Producción**.
4. En el mismo menú abre **Authentication**, pulsa **Comenzar**, entra a la pestaña **Método de acceso** (*Sign-in method*), elige **Correo electrónico/contraseña** y activa **solo la primera opción** (la de correo y contraseña, no la del enlace). Guarda.
5. Pulsa el engranaje ⚙ junto a *Descripción general del proyecto* y elige **Configuración del proyecto**. En la pestaña **General** anota dos datos:
   - **ID del proyecto** (*Project ID*), por ejemplo `biblioteca-clase-1a2b3`.
   - **Clave de API web** (*Web API Key*). Si no la ves, baja a *Tus apps*, pulsa el icono `</>` (Web), registra una app llamada `biblioteca-web` sin activar Hosting y copia el valor `apiKey` que aparece.
6. En la pestaña **Cuentas de servicio** (*Service accounts*) pulsa **Generar nueva clave privada**. Se descarga un archivo `.json`. Guárdalo, lo usaremos en el paso 1.7.

> **La llave privada es como la contraseña maestra de tu base de datos**
>
> Quien tenga ese archivo `.json` puede leer y borrar todo. **Nunca** lo subas a GitHub, no lo envíes por chat ni lo pegues en capturas. En este proyecto lo guardaremos en una carpeta que Git ignora.

> **¿Y las reglas de seguridad de Firestore?**
>
> Elegimos *modo producción*, que bloquea el acceso desde navegadores. No nos afecta: nuestro servidor usa la llave privada (Admin SDK), que tiene acceso total. Los usuarios nunca hablan directo con Firestore, solo con nuestro servidor Django.

1.4 Preparar la carpeta y el entorno virtual

Un **entorno virtual** es una carpeta (`.venv`) donde se instalan las librerías *solo para este proyecto*, sin mezclarlas con las de otros proyectos.

Abre una terminal (en VS Code: **Terminal → Nueva terminal**) y crea la carpeta del proyecto:

```
TERMINAL
mkdir biblioteca
cd biblioteca
```

Comprueba tu versión de Python (necesitas **3.12 o superior**):

```
TERMINAL
python3 --version
```

Crea el entorno virtual:

```
TERMINAL
python3 -m venv .venv
```

Actívalo. **Tienes que hacerlo cada vez que abras una terminal nueva** para trabajar en el proyecto:

TERMINAL

```
source .venv/bin/activate
```

Si usas Windows

En lugar de `python3` escribe `python`, y para activar el entorno usa uno de estos comandos:

PowerShell: `.venv\Scripts\Activate.ps1` · CMD: `.venv\Scripts\activate.bat`

Si PowerShell dice que la ejecución de scripts está deshabilitada, ejecuta una vez `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` y vuelve a intentar.

✔ Verificación

Al comienzo de la línea de tu terminal debe aparecer `(.venv)`. Si no aparece, el entorno no está activo y los siguientes comandos fallarán.

1.5 Instalar las dependencias

Crea el archivo `requirements.txt` en la raíz de la carpeta `biblioteca`. Lista las librerías con la **versión exacta** que usa esta guía, para que tu proyecto se comporte igual que el de los ejemplos:

requirements.txt — ARCHIVO NUEVO

```
Django==6.1.2
firebase-admin==7.7.0
python-dotenv==1.2.4
requests==2.34.2
plotly==7.1.0
```

Librería	Para qué sirve
`Django`	El framework web
`firebase-admin`	Hablar con Firestore y Authentication desde el servidor
`python-dotenv`	Leer secretos desde el archivo `.env`
`requests`	Hacer la llamada HTTP que verifica la contraseña
`plotly`	Dibujar las gráficas del dashboard (clase 4)

Instálalas (tarda uno o dos minutos):

TERMINAL

```
pip install -r requirements.txt
```

1.6 Crear el proyecto Django

```
TERMINAL
django-admin startproject config .
```

Fíjate en el punto final: significa "créalo **aquí**, en esta carpeta". Si lo olvidas, Django crea una carpeta extra y todo queda un nivel más profundo de lo que indica esta guía. El nombre `config` es la carpeta de configuración del proyecto.

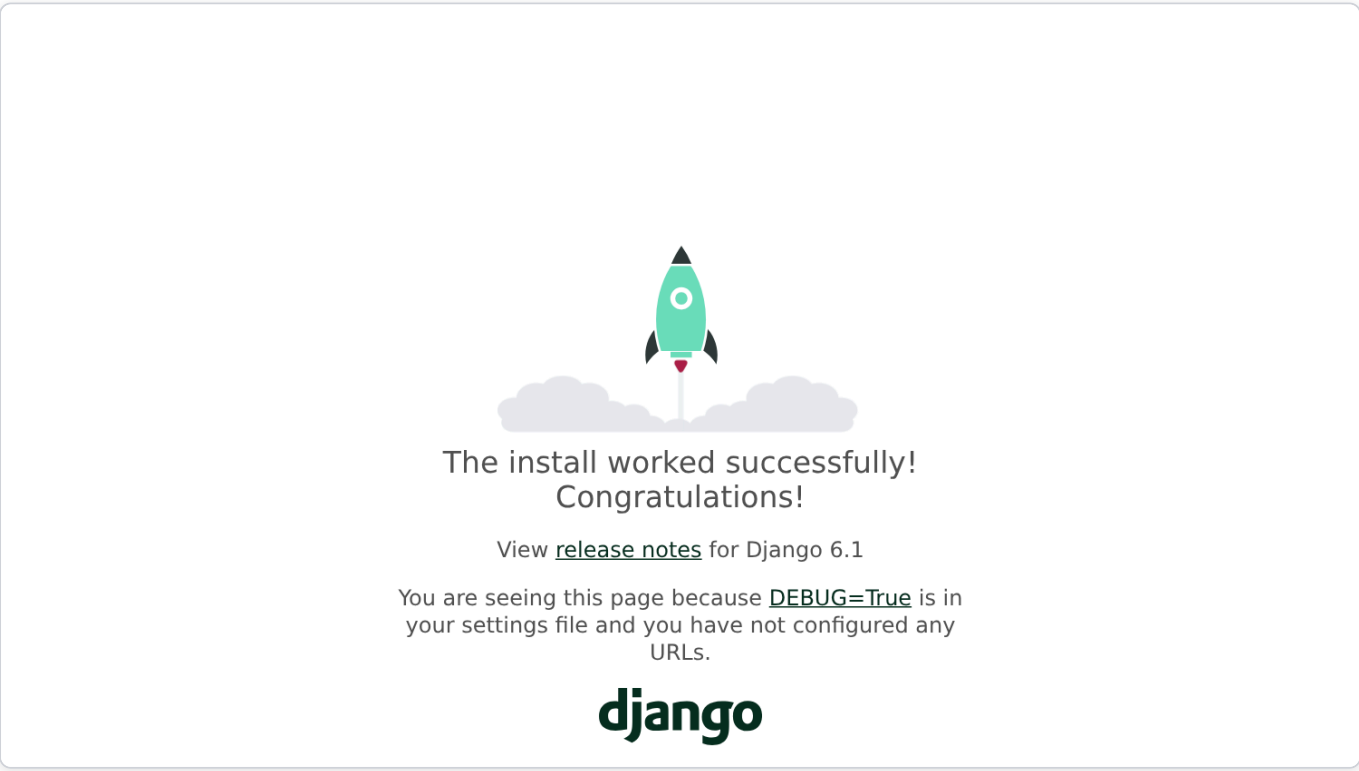
Ahora tu carpeta se ve así:

```
EJEMPLO
biblioteca/
├── .venv/              (el entorno virtual)
├── config/             (configuración del proyecto)
│   ├── __init__.py
│   ├── asgi.py
│   ├── settings.py     (la configuración principal)
│   ├── urls.py         (las rutas del proyecto)
│   └── wsgi.py
├── manage.py           (el comando para controlar el proyecto)
└── requirements.txt
```

Arranca el servidor de desarrollo:

```
TERMINAL
python manage.py runserver
```

Abre http://127.0.0.1:8000 en el navegador. Debes ver la página del cohete:



Página de bienvenida de Django: el servidor funciona

Detén el servidor con **Ctrl + C**. Luego crea las tablas internas de Django con:

TERMINAL

```
python manage.py migrate
```

SALIDA ESPERADA

```
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
```

> **¿Para qué migrar si usamos Firebase?**
>
> Django guarda las **sesiones** (quién inició sesión) en una pequeña base de datos local, el archivo `db.sqlite3`. `migrate` crea esas tablas. Nuestros datos de verdad (libros, préstamos, usuarios) irán en Firestore.

1.7 Secretos: `.env`, la llave y `.gitignore`

Los secretos **no se escriben en el código**: viven en un archivo `.env` que no se comparte. Crea `.env` en la raíz (junto a `manage.py`) y reemplaza los valores por **los tuyos** (los que anotaste en el paso 1.3):

.env — ARCHIVO NUEVO

```
FIREBASE_PROJECT_ID=tu-id-de-proyecto
FIREBASE_WEB_API_KEY=tu-clave-de-api-web
FIREBASE_CREDENTIALS_PATH=firebase/serviceAccountKey.json
```

Crea la carpeta `firebase` y mueve ahí la llave privada que descargaste, cambiándole el nombre a `serviceAccountKey.json`:

TERMINAL

```
mkdir -p firebase
```

Tu carpeta debe contener ahora `firebase/serviceAccountKey.json`.

Por último, crea el archivo `.gitignore`. Le dice a Git qué archivos **nunca** debe subir:

.gitignore — ARCHIVO NUEVO

```
.venv/
.env
firebase/
db.sqlite3
__pycache__/
*.pyc
```

> **✔ Verificación**
>
> Si usas Git, ejecuta `git status`: no debe aparecer `.env` ni la carpeta `firebase/`. Si aparecen, revisa que `.gitignore` esté en la raíz y se llame exactamente así.

1.8 Organizar el proyecto por features

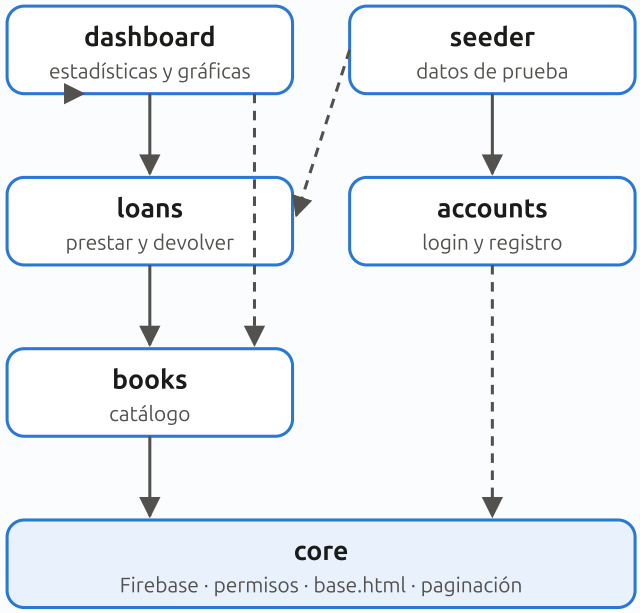
La mayoría de tutoriales organizan el código **por tipo de archivo**: una carpeta con todas las vistas, otra con todas las plantillas… Cuando el proyecto crece, para cambiar algo del préstamo de libros tienes que abrir cinco carpetas distintas.

Nosotros usaremos **arquitectura por features** (*feature-driven*): todo lo que pertenece a una funcionalidad vive **junto**, en una sola carpeta.

EJEMPLO

```
biblioteca/
├── config/              ← configuración global del proyecto
├── core/                ← lo compartido (conexión a Firebase, base.html, permisos…)
└── features/
    ├── accounts/        ← login, registro y logout
    ├── books/           ← catálogo de libros
    ├── loans/           ← prestar, devolver y cancelar
    ├── dashboard/       ← estadísticas y gráficas
    └── seeder/          ← datos de prueba
```

Cada feature es una **app de Django** con sus propios `views.py`, `urls.py`, `services.py`, `templates/` y `tests.py`. La regla de oro es la dirección de las dependencias:



Cómo leerlo

Una flecha significa "usa a".
Todas las features usan a **core**.
loans usa a **books**, nunca al revés.
dashboard y **seeder** solo leen datos de las demás: nadie depende de ellos.

Figura: dependencias entre las features (siempre en un solo sentido)

Una feature puede usar `core`, y algunas usan a otras en **un solo sentido** (por ejemplo, `loans` usa `books`). Nunca al revés, así evitamos enredos.

Hoy creamos `core` y `accounts`. Las demás features las crearemos en cada clase. Crea las carpetas y los archivos vacíos `__init__.py` (le indican a Python que la carpeta es un paquete):

TERMINAL

```
mkdir -p core/templates/core core/static/core/css
mkdir -p features/accounts/templates/accounts features/accounts/management/commands
touch core/__init__.py features/__init__.py features/accounts/__init__.py
touch features/accounts/management/__init__.py features/accounts/management/commands/__init__.py
```

> **Si usas Windows**
>
> PowerShell no tiene `touch`. Crea las carpetas con `mkdir core\templates\core` (crea las intermedias solas) y cada archivo vacío con `ni core\__init__.py`. También puedes hacerlo con el explorador de VS Code: clic derecho → *Nuevo archivo*.

1.9 Configurar `settings.py`

Abre `config/settings.py`. Vamos a hacer **seis modificaciones pequeñas**. En cada una: busca el texto del primer bloque (está tal cual en tu archivo) y reemplázalo por el segundo.

Modificación 1 · Importaciones. Necesitamos `os` (leer variables), `sys` (detectar los tests) y `dotenv` (leer el `.env`):

MODIFICA `config/settings.py`

BUSCA este texto (está tal cual en el archivo):

```
from pathlib import Path
```

y REEMPLÁZALO por:

```
import os
import sys
from pathlib import Path

from dotenv import load_dotenv
```

Modificación 2 · Cargar el `.env`. Justo después de definir `BASE_DIR`:

MODIFICA `config/settings.py`

BUSCA este texto (está tal cual en el archivo):

```
BASE_DIR = Path(__file__).resolve().parent.parent
```

y REEMPLÁZALO por:

```
BASE_DIR = Path(__file__).resolve().parent.parent

load_dotenv(BASE_DIR / '.env')
```

Modificación 3 · Registrar nuestras apps. Django solo reconoce las apps que aparecen en `INSTALLED_APPS`:

MODIFICA `config/settings.py`

BUSCA este texto (está tal cual en el archivo):

```
    'django.contrib.staticfiles',
]
```

y REEMPLÁZALO por:

```
    'django.contrib.staticfiles',
    'core',
    'features.accounts',
]
```

Modificación 4 · Un procesador de contexto. Hace que la variable `current_user` esté disponible en **todas** las plantillas:

MODIFICA `config/settings.py`

BUSCA este texto (está tal cual en el archivo):

```
                'django.contrib.messages.context_processors.messages',
```

y REEMPLÁZALO por:

```
                'django.contrib.messages.context_processors.messages',
                'core.context_processors.session_user',
```

Modificación 5 · Idioma y zona horaria.

MODIFICA `config/settings.py`

BUSCA este texto (está tal cual en el archivo):

```
LANGUAGE_CODE = 'en-us'

TIME_ZONE = 'UTC'
```

y REEMPLÁZALO por:

```
LANGUAGE_CODE = 'es-co'

TIME_ZONE = 'America/Bogota'
```

Modificación 6 · Configuración propia. Ve al **final** del archivo, busca el bloque `MAILERS` y agrega después todo lo nuevo:

MODIFICA `config/settings.py`

BUSCA este texto (está tal cual en el archivo):

```
        'BACKEND': 'django.core.mail.backends.console.EmailBackend',
    },
}
```

y REEMPLÁZALO por:

```
        'BACKEND': 'django.core.mail.backends.console.EmailBackend',
    },
}


# Firebase
# Los valores viven en el archivo .env (no se sube a git).
FIREBASE_PROJECT_ID = os.environ.get('FIREBASE_PROJECT_ID', '')
FIREBASE_WEB_API_KEY = os.environ.get('FIREBASE_WEB_API_KEY', '')
FIREBASE_CREDENTIALS_PATH = BASE_DIR / os.environ.get(
    'FIREBASE_CREDENTIALS_PATH', 'firebase/serviceAccountKey.json'
)

# Reglas de negocio de la biblioteca
LOAN_DAYS = 14          # días de plazo de un préstamo
MAX_ACTIVE_LOANS = 3    # préstamos activos simultáneos por estudiante

# Login
LOGIN_URL = 'accounts:login'

# Paginación
PAGE_SIZE = 10

# True cuando se corre `python manage.py test`: los tests nunca deben tocar Firestore
TESTING = len(sys.argv) > 1 and sys.argv[1] == 'test'
```

> **No toques `SECRET_KEY`**
>
> Django generó una clave secreta única para tu proyecto. Déjala como está. Algunas de las constantes nuevas (`LOAN_DAYS` , `PAGE_SIZE` …) las usaremos en clases posteriores.

1.10 Las rutas del proyecto

`config/urls.py` es la "puerta de entrada" de las direcciones. Cada feature tendrá su propio `urls.py` y aquí solo las **incluimos**. Reemplaza las importaciones y la lista `urlpatterns`:

MODIFICA `config/urls.py`

BUSCA este texto (está tal cual en el archivo):

```
from django.contrib import admin
from django.urls import path

urlpatterns = [
    path('admin/', admin.site.urls),
]
```

y REEMPLÁZALO por:

```
from django.urls import include, path

urlpatterns = [
    path('', include('core.urls')),
    path('', include('features.accounts.urls')),
]
```

Quitamos `admin/` porque nuestros usuarios viven en Firebase, no en la base de datos de Django.

1.11 `core`: lo que comparten todas las features

Crearemos estos archivos en `core/`:

EJEMPLO

```
core/
├── apps.py                 configuración de la app
├── firebase.py             la conexión única a Firebase
├── context_processors.py   expone current_user a las plantillas
├── decorators.py           permisos: ¿quién puede entrar a cada página?
├── views.py  urls.py       la página de inicio
├── static/core/css/site.css
└── templates/core/         base.html, _form.html, home.html
```

`core/apps.py`

Toda app de Django tiene un `AppConfig`. Aquí solo le decimos cómo se llama:

core/apps.py — ARCHIVO NUEVO

```
from django.apps import AppConfig


class CoreConfig(AppConfig):
    name = 'core'
```

`core/firebase.py` · la conexión a Firebase

`get_db()` devuelve el cliente de Firestore. La primera vez que se llama, inicia Firebase leyendo tu llave privada; las siguientes reutiliza esa conexión. Fíjate en la guarda de `TESTING`: impide que los tests escriban en tu base de datos real por accidente.

core/firebase.py — ARCHIVO NUEVO

```python
"""Conexión única a Firebase (Firestore + Auth) para todo el proyecto."""
import firebase_admin
from django.conf import settings
from firebase_admin import credentials, firestore


def get_db():
    """Devuelve el cliente de Firestore, inicializando Firebase la primera vez."""
    if settings.TESTING:
        raise RuntimeError('Los tests no deben tocar Firestore: usa mock.patch sobre el servicio.')
    try:
        firebase_admin.get_app()
    except ValueError:
        cred = credentials.Certificate(str(settings.FIREBASE_CREDENTIALS_PATH))
        firebase_admin.initialize_app(cred)
    return firestore.client()
```

`core/context_processors.py`

Una función que Django ejecuta en **cada** página y cuyo resultado queda disponible en las plantillas. Así cualquier plantilla puede escribir `{{ current_user.name }}`:

core/context_processors.py — ARCHIVO NUEVO

```python
def session_user(request):
    """Pone `current_user` disponible en todos los templates."""
    return {'current_user': request.session.get('user')}
```

`core/decorators.py` · permisos

Un **decorador** es una función que envuelve a otra. Estos tres protegen las vistas: `@login_required` exige haber iniciado sesión y `@librarian_required` / `@student_required` exigen un rol. Los usaremos así:

EJEMPLO

```python
@librarian_required        # solo bibliotecarios
def dashboard(request):
    ...
```

core/decorators.py — ARCHIVO NUEVO

```python
from functools import wraps

from django.contrib import messages
from django.contrib.auth.views import redirect_to_login
from django.shortcuts import redirect
from django.urls import reverse

LIBRARIAN = 'librarian'
```

```python
STUDENT = 'student'


def login_required(view):
    """Exige que haya un usuario en la sesión."""
    @wraps(view)
    def wrapper(request, *args, **kwargs):
        if not request.session.get('user'):
            return redirect_to_login(request.get_full_path(), reverse('accounts:login'))
        return view(request, *args, **kwargs)
    return wrapper


def role_required(role):
    """Exige estar logueado y tener el rol indicado."""
    def decorator(view):
        @wraps(view)
        @login_required
        def wrapper(request, *args, **kwargs):
            if request.session['user']['role'] != role:
                messages.error(request, 'No tienes permiso para ver esa página.')
                return redirect('books:list')
            return view(request, *args, **kwargs)
        return wrapper
    return decorator


librarian_required = role_required(LIBRARIAN)
student_required = role_required(STUDENT)
```

> **Qué pasa cuando alguien no tiene permiso**
>
> Si no ha iniciado sesión, lo mandamos al login (recordando a dónde quería ir con `?next=`). Si inició sesión pero su rol no sirve, mostramos el mensaje *"No tienes permiso para ver esa página"* y lo enviamos al catálogo (que construiremos en la clase 2).

`core/views.py` y `core/urls.py` · página de inicio provisional

Por ahora la página de inicio solo confirma que iniciaste sesión. En la clase 2 la reemplazaremos por una redirección según el rol.

core/views.py — ARCHIVO NUEVO

```python
from django.shortcuts import render

from .decorators import login_required


@login_required
def home(request):
    """Página de inicio provisional: confirma que el login funcionó."""
    return render(request, 'core/home.html')
```

El archivo `urls.py` conecta la dirección `/` con esa vista. `app_name = 'core'` crea el **espacio de nombres** `core:`, así en las plantillas escribiremos `{% url 'core:home' %}` sin riesgo de confundirnos con otra `home`:

core/urls.py — ARCHIVO NUEVO

```python
from django.urls import path

from . import views

app_name = 'core'

urlpatterns = [
    path('', views.home, name='home'),
]
```

Plantillas de `core`

`base.html` es la **plantilla madre**: todas las páginas la extienden y solo rellenan el bloque `content`. Incluye la barra de navegación, los mensajes (*flash*) y los estilos. Usamos **Pico CSS** (una hoja de estilos lista para usar, cargada desde internet) más nuestro propio `site.css`.

Por ahora el menú solo tiene *Ingresar*, *Registrarse* y *Salir*; iremos agregando enlaces en cada clase.

core/templates/core/base.html — ARCHIVO NUEVO

```
{% load static %}<!DOCTYPE html>
<html lang="es" data-theme="light">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>{% block title %}Biblioteca{% endblock %} · Biblioteca TdeA</title>
  <link rel="stylesheet" href="https://cdn.jsdelivr.net/npm/@picocss/pico@2.0.6/css/pico.min.css">
  <link rel="stylesheet" href="{% static 'core/css/site.css' %}">
</head>
<body>
  <header class="container">
    <nav>
      <ul><li><strong><a href="{% url 'core:home' %}" class="contrast">📚 Biblioteca</a></strong></li></ul>
      <ul>
        {% if current_user %}
          <li>
            <form method="post" action="{% url 'accounts:logout' %}">{% csrf_token %}
              <button class="outline secondary nav-logout">
                Salir ({{ current_user.name }}<span class="nav-role"> · {% if current_user.role == 'librarian'
%}bibliotecario{% else %}estudiante{% endif %}</span>)
              </button>
            </form>
          </li>
        {% else %}
          <li><a href="{% url 'accounts:login' %}">Ingresar</a></li>
          <li><a href="{% url 'accounts:register' %}">Registrarse</a></li>
        {% endif %}
      </ul>
    </nav>
  </header>
  <main class="container">
    {% for message in messages %}
      <article class="flash{% if message.tags == 'error' %} flash-error{% endif %}">
        {{ message }}
      </article>
    {% endfor %}
    {% block content %}{% endblock %}
  </main>
```

```
</body>
</html>
```

`_form.html` dibuja cualquier formulario de Django campo por campo, con sus errores. Lo reutilizarán el login, el registro y el formulario de libros. (El guion bajo inicial indica que es un *fragmento*, no una página completa.)

core/templates/core/_form.html — ARCHIVO NUEVO

```
{% for error in form.non_field_errors %}<p><small class="field-error">{{ error }}</small></p>{% endfor %}
{% for field in form %}
  <label>{{ field.label }}
    {{ field }}
    {% for error in field.errors %}<small class="field-error">{{ error }}</small>{% endfor %}
  </label>
{% endfor %}
```

Y la página de inicio provisional:

core/templates/core/home.html — ARCHIVO NUEVO

```
{% extends 'core/base.html' %}
{% block title %}Inicio{% endblock %}
{% block content %}
<h1>Hola, {{ current_user.name }}</h1>
<p>Iniciaste sesión como <strong>{% if current_user.role == 'librarian' %}bibliotecario{% else %}estudiante{%
endif %}</strong>.</p>
<p>Tu sesión funciona. En la clase 2 esta página será el catálogo de libros.</p>
{% endblock %}
```

`core/static/core/css/site.css` · estilos

Creamos el archivo con la primera parte de nuestros estilos: la navegación y los mensajes. En cada clase agregaremos secciones al final.

core/static/core/css/site.css — ARCHIVO NUEVO

```
/* Estilos propios sobre Pico CSS. Breakpoints: 768px (tablet) y 480px (móvil). */

/* --- Navegación ------------------------------------------------------------ */
nav { flex-wrap: wrap; row-gap: .25rem; }
nav ul { flex-wrap: wrap; }
nav form { margin: 0; }
.nav-logout { margin: 0; width: auto; padding: .25rem .8rem; }

/* --- Mensajes y formularios ------------------------------------------------ */
.flash { padding: .7rem 1rem; margin-bottom: 1rem; border-left: 4px solid #0ca30c; }
.flash-error { border-left-color: #d03b3b; }
.field-error { color: #d03b3b; }
.auth-card { max-width: 28rem; margin: 2rem auto; }
.form-card { max-width: 36rem; margin: 1rem auto; }
```

1.12 `accounts`: login, registro y logout

Esta feature contiene todo lo relacionado con usuarios. Crea estos archivos en `features/accounts/`:

EJEMPLO

```
features/accounts/
├── apps.py  forms.py  services.py  views.py  urls.py
├── management/commands/create_user.py     (sección 1.13)
├── templates/accounts/login.html  register.html
└── tests.py                               (sección 1.15)
```

`apps.py`

features/accounts/apps.py — ARCHIVO NUEVO

```python
from django.apps import AppConfig


class AccountsConfig(AppConfig):
    name = 'features.accounts'
```

El `name` es la ruta completa del paquete. Por eso, en `INSTALLED_APPS`, escribimos `'features.accounts'`.

`forms.py` · los formularios

Un **formulario de Django** declara los campos, los valida y genera el HTML. `RegisterForm.clean()` es una validación que involucra dos campos a la vez: que las contraseñas coincidan.

features/accounts/forms.py — ARCHIVO NUEVO

```python
from django import forms


class LoginForm(forms.Form):
    email = forms.EmailField(label='Correo')
    password = forms.CharField(label='Contraseña', widget=forms.PasswordInput)


class RegisterForm(forms.Form):
    name = forms.CharField(label='Nombre', max_length=80)
    email = forms.EmailField(label='Correo')
    password = forms.CharField(label='Contraseña', min_length=6, widget=forms.PasswordInput)
    password2 = forms.CharField(label='Repite la contraseña', widget=forms.PasswordInput)

    def clean(self):
        data = super().clean()
        if data.get('password') and data.get('password') != data.get('password2'):
            self.add_error('password2', 'Las contraseñas no coinciden.')
        return data
```

`services.py` · hablar con Firebase

Es el archivo más importante de la feature. Cada función hace **una** cosa:

Función	Qué hace
`sign_in(email, password)`	Pregunta a Firebase (API REST) si el correo y la contraseña son correctos. Si lo son, verifica el token y devuelve los datos de sesión

Función	Qué hace
`get_session_user(uid, email, name)`	Lee el **rol** del usuario en la colección `users` de Firestore (si no existe, lo crea como estudiante)
`create_user(name, email, password, role)`	Crea la cuenta en Authentication y su perfil con rol en Firestore
`register_student(...)`	El registro público: siempre crea un estudiante
`list_students()`	Lista los estudiantes (la usará el *seeder*)

`AUTH_ERRORS` traduce los códigos de error de Firebase (como `INVALID_LOGIN_CREDENTIALS`) a mensajes que entiende un usuario.

features/accounts/services.py — ARCHIVO NUEVO

```python
"""Usuarios: autenticación con Firebase Auth y perfil/rol en Firestore."""
import requests
from django.conf import settings
from firebase_admin import auth, firestore
from google.cloud.firestore_v1.base_query import FieldFilter

from core.firebase import get_db

SIGN_IN_URL = 'https://identitytoolkit.googleapis.com/v1/accounts:signInWithPassword'

# Mensajes de error de la REST API de Firebase traducidos
AUTH_ERRORS = {
    'INVALID_LOGIN_CREDENTIALS': 'Correo o contraseña incorrectos.',
    'INVALID_PASSWORD': 'Correo o contraseña incorrectos.',
    'EMAIL_NOT_FOUND': 'Correo o contraseña incorrectos.',
    'USER_DISABLED': 'Esta cuenta está deshabilitada.',
    'TOO_MANY_ATTEMPTS_TRY_LATER': 'Demasiados intentos. Intenta más tarde.',
}


class AuthError(Exception):
    """Error de autenticación con un mensaje listo para mostrar al usuario."""


def sign_in(email, password):
    """Valida correo y contraseña contra Firebase Auth y devuelve los datos de sesión.

    La contraseña solo se puede verificar con la REST API (el Admin SDK no puede),
    por eso usamos `requests` aquí.
    """
    response = requests.post(
        SIGN_IN_URL,
        params={'key': settings.FIREBASE_WEB_API_KEY},
        json={'email': email, 'password': password, 'returnSecureToken': True},
        timeout=10,
    )
    if response.status_code != 200:
        code = response.json().get('error', {}).get('message', '')
        raise AuthError(AUTH_ERRORS.get(code.split(' ')[0], 'No se pudo iniciar sesión.'))

    # Verificamos el token con el Admin SDK en lugar de confiar a ciegas en la respuesta.
    get_db()
    decoded = auth.verify_id_token(response.json()['idToken'], clock_skew_seconds=10)
    return get_session_user(decoded['uid'], email, decoded.get('name', ''))
```

```python
def get_session_user(uid, email, name=''):
    """Lee el rol del usuario en Firestore (lo crea como estudiante si no existe)."""
    ref = get_db().collection('users').document(uid)
    snapshot = ref.get()
    if snapshot.exists:
        data = snapshot.to_dict()
    else:
        data = {'email': email, 'name': name or email.split('@')[0], 'role': 'student'}
        ref.set({**data, 'created_at': firestore.SERVER_TIMESTAMP})
    return {'uid': uid, 'email': data['email'], 'name': data['name'], 'role': data['role']}


def create_user(name, email, password, role):
    """Crea la cuenta en Firebase Auth y su perfil (con el rol) en Firestore."""
    get_db()
    try:
        user = auth.create_user(email=email, password=password, display_name=name)
    except auth.EmailAlreadyExistsError:
        raise AuthError('Ya existe una cuenta con ese correo.')
    except ValueError as exc:
        raise AuthError(str(exc))
    get_db().collection('users').document(user.uid).set(
        {'email': email, 'name': name, 'role': role, 'created_at': firestore.SERVER_TIMESTAMP}
    )
    return user.uid


def register_student(name, email, password):
    """Registro público: siempre crea un estudiante."""
    return create_user(name, email, password, role='student')


def list_students():
    """Perfiles con rol estudiante: [{'uid', 'email', 'name', ...}]."""
    docs = get_db().collection('users').where(filter=FieldFilter('role', '==', 'student')).stream()
    return [{'uid': d.id, **d.to_dict()} for d in docs]
```

> **¿Por qué el rol no va en Authentication?**
>
> Authentication solo sabe de correos y contraseñas. El **rol** (`student` o `librarian`) lo guardamos en nuestro propio documento `users/{uid}` en Firestore. Así podemos cambiarlo cuando queramos sin tocar la cuenta.

`views.py` · las vistas

Tres vistas, cada una muy corta porque el trabajo pesado está en `services.py`:

- `login_view`: si el formulario es válido, llama a `services.sign_in`. Si sale bien, guarda al usuario en `request.session['user']`; si no, muestra el error en el formulario.
- `register_view`: crea el estudiante y manda al login.
- `logout_view`: borra la sesión. Solo acepta `POST` (`@require_POST`) para que nadie pueda cerrarte la sesión con un simple enlace.

Dos detalles de seguridad: `request.session.cycle_key()` cambia el identificador de sesión al entrar, y el parámetro `next` solo se acepta si empieza por `/` (evita redirecciones a sitios externos).

features/accounts/views.py — ARCHIVO NUEVO

```python
from django.contrib import messages
from django.shortcuts import redirect, render
from django.views.decorators.http import require_POST

from . import services
from .forms import LoginForm, RegisterForm
from .services import AuthError


def login_view(request):
    if request.session.get('user'):
        return redirect('core:home')
    form = LoginForm(request.POST or None)
    if request.method == 'POST' and form.is_valid():
        try:
            user = services.sign_in(form.cleaned_data['email'], form.cleaned_data['password'])
        except AuthError as exc:
            form.add_error(None, str(exc))
        else:
            request.session.cycle_key()
            request.session['user'] = user
            messages.success(request, f"¡Hola, {user['name']}!")
            next_url = request.GET.get('next', '')
            return redirect(next_url if next_url.startswith('/') else 'core:home')
    return render(request, 'accounts/login.html', {'form': form})


def register_view(request):
    form = RegisterForm(request.POST or None)
    if request.method == 'POST' and form.is_valid():
        try:
            services.register_student(
                form.cleaned_data['name'], form.cleaned_data['email'], form.cleaned_data['password']
            )
        except AuthError as exc:
            form.add_error(None, str(exc))
        else:
            messages.success(request, 'Cuenta creada. Ya puedes iniciar sesión.')
            return redirect('accounts:login')
    return render(request, 'accounts/register.html', {'form': form})


@require_POST
def logout_view(request):
    request.session.flush()
    return redirect('accounts:login')
```

urls.py

features/accounts/urls.py — ARCHIVO NUEVO

```python
from django.urls import path

from . import views

app_name = 'accounts'

urlpatterns = [
    path('login/', views.login_view, name='login'),
    path('register/', views.register_view, name='register'),
```

```
    path('logout/', views.logout_view, name='logout'),
]
```

Plantillas de `accounts`

features/accounts/templates/accounts/login.html — ARCHIVO NUEVO

```
{% extends 'core/base.html' %}
{% block title %}Ingresar{% endblock %}
{% block content %}
<article class="auth-card">
  <h2>Iniciar sesión</h2>
  <form method="post">{% csrf_token %}
    {% include 'core/_form.html' %}
    <button type="submit">Ingresar</button>
  </form>
  <p>¿No tienes cuenta? <a href="{% url 'accounts:register' %}">Regístrate como estudiante</a></p>
</article>
{% endblock %}
```

features/accounts/templates/accounts/register.html — ARCHIVO NUEVO

```
{% extends 'core/base.html' %}
{% block title %}Registro{% endblock %}
{% block content %}
<article class="auth-card">
  <h2>Crear cuenta de estudiante</h2>
  <form method="post">{% csrf_token %}
    {% include 'core/_form.html' %}
    <button type="submit">Registrarme</button>
  </form>
  <p>¿Ya tienes cuenta? <a href="{% url 'accounts:login' %}">Ingresa</a></p>
</article>
{% endblock %}
```

1.13 Crear tus usuarios con un comando

Necesitas al menos un **bibliotecario** y un **estudiante** para probar. Para no dejar contraseñas escritas en ningún archivo, creamos un **comando de administración** propio: `python manage.py create_user`. Pide la contraseña por teclado (sin mostrarla) y no la guarda en ningún lado.

Un comando de Django es un archivo dentro de `management/commands/`; el nombre del archivo es el nombre del comando.

features/accounts/management/commands/create_user.py — ARCHIVO NUEVO

```
"""Crea un usuario (estudiante o bibliotecario) en Firebase.

Uso:  python manage.py create_user ana@example.com "Ana Torres" --role librarian
La contraseña se pide por teclado: no queda guardada en ningún archivo.
"""
from getpass import getpass

from django.core.management.base import BaseCommand, CommandError

from features.accounts import services
```

```
class Command(BaseCommand):
    help = 'Crea un usuario en Firebase Auth con su rol en Firestore.'

    def add_arguments(self, parser):
        parser.add_argument('email')
        parser.add_argument('name', help='Nombre completo (entre comillas si tiene espacios).')
        parser.add_argument('--role', choices=['student', 'librarian'], default='student')

    def handle(self, *args, **options):
        password = getpass('Contraseña (mínimo 6 caracteres): ')
        if getpass('Repite la contraseña: ') != password:
            raise CommandError('Las contraseñas no coinciden.')
        try:
            services.create_user(options['name'], options['email'], password, options['role'])
        except services.AuthError as exc:
            raise CommandError(str(exc))
        self.stdout.write(self.style.SUCCESS(f"Usuario {options['email']} creado con rol {options['role']}."))
```

Ejecútalo para crear tu bibliotecario (usa **tu correo real** y una contraseña de mínimo 6 caracteres):

TERMINAL

```
python manage.py create_user tucorreo@gmail.com "Tu Nombre" --role librarian
```

SALIDA ESPERADA

```
Contraseña (mínimo 6 caracteres):
Repite la contraseña:
Usuario tucorreo@gmail.com creado con rol librarian.
```

Crea también **dos estudiantes** (en la clase 2 repartiremos préstamos de prueba entre los estudiantes que existan, y con dos se ven mejor las gráficas):

TERMINAL

```
python manage.py create_user estudiante1@example.com "Estudiante Uno"
python manage.py create_user estudiante2@example.com "Estudiante Dos"
```

> **✔ Verificación en la consola de Firebase**
>
> Abre **Authentication → Usuarios**: deben aparecer los tres correos. Luego abre **Firestore Database → Datos**: debe existir la colección `users` con tres documentos, cada uno con los campos `email`, `name`, `role` y `created_at`.

1.14 Probar en el navegador

TERMINAL

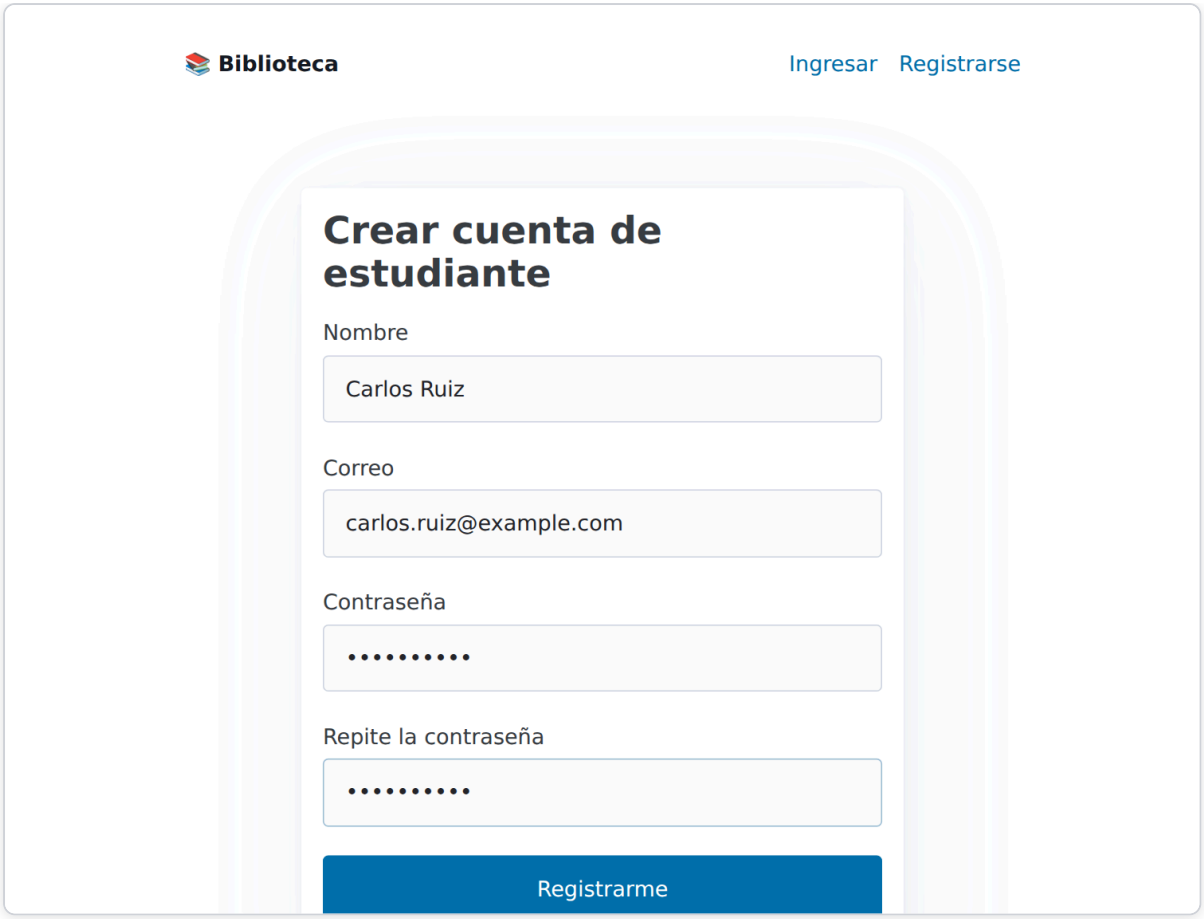
```
python manage.py runserver
```

1. Abre http://127.0.0.1:8000/login/.
2. Escribe un correo o una contraseña incorrectos: debe aparecer *"Correo o contraseña incorrectos."*

3. Ingresa con tu bibliotecario: llegas a la página de inicio con tu nombre y rol.
4. Pulsa **Salir** y comprueba que vuelves al login.
5. Abre http://127.0.0.1:8000/register/ y registra un estudiante nuevo.



Pantalla de inicio de sesión



Pantalla de registro de estudiantes



Página de inicio provisional tras iniciar sesión

1.15 Tests de la clase 1

Escribir pruebas automáticas te permite cambiar código mañana sin miedo a romper lo de hoy. Reglas de este proyecto: **los tests nunca tocan Firebase**. Usamos `mock.patch` para *reemplazar* temporalmente una función real (como `sign_in`) por una falsa que devuelve lo que necesitamos.

Primero, un pequeño archivo de ayuda que usarán los tests de todas las features. `SessionTestCase.login_as()` simula un usuario con sesión iniciada, sin pasar por Firebase:

core/testing.py — ARCHIVO NUEVO

```python
"""Utilidades compartidas por los tests de todas las features."""
from django.test import TestCase

LIBRARIAN = {'uid': 'lib1', 'email': 'bib@example.com', 'name': 'Bibliotecaria', 'role': 'librarian'}
STUDENT = {'uid': 'est1', 'email': 'est@example.com', 'name': 'Estudiante', 'role': 'student'}


class SessionTestCase(TestCase):
    """TestCase con helpers para iniciar sesión sin pasar por Firebase."""

    def login_as(self, user):
        session = self.client.session
        session['user'] = user
        session.save()
```

Ahora los tests de `accounts`. Léelos con calma: cada método de prueba es una frase que describe un comportamiento (`test_successful_login_stores_user_in_session`).

features/accounts/tests.py — ARCHIVO NUEVO

```python
from io import StringIO
from unittest import mock

from django.core.management import call_command
from django.core.management.base import CommandError
from django.test import SimpleTestCase

from core.testing import STUDENT, SessionTestCase

from . import services
from .forms import RegisterForm
from .services import AuthError


class SignInTests(SimpleTestCase):
    def test_firebase_error_is_translated(self):
        response = mock.Mock(status_code=400)
        response.json.return_value = {'error': {'message': 'INVALID_LOGIN_CREDENTIALS'}}
        with mock.patch('features.accounts.services.requests.post', return_value=response):
            with self.assertRaisesMessage(AuthError, 'Correo o contraseña incorrectos.'):
                services.sign_in('a@b.com', 'mala')

    def test_unknown_firebase_error_gets_generic_message(self):
        response = mock.Mock(status_code=400)
        response.json.return_value = {'error': {'message': 'ALGO_RARO'}}
        with mock.patch('features.accounts.services.requests.post', return_value=response):
            with self.assertRaisesMessage(AuthError, 'No se pudo iniciar sesión.'):
                services.sign_in('a@b.com', 'x')


class RegisterFormTests(SimpleTestCase):
    def test_passwords_must_match(self):
        form = RegisterForm({'name': 'Ana', 'email': 'a@b.com', 'password': 'abcdef', 'password2': 'abcdeX'})
        self.assertFalse(form.is_valid())
        self.assertIn('password2', form.errors)

    def test_password_min_length(self):
        form = RegisterForm({'name': 'Ana', 'email': 'a@b.com', 'password': '123', 'password2': '123'})
        self.assertFalse(form.is_valid())
```

```
class LoginViewTests(SessionTestCase):
    def test_successful_login_stores_user_in_session(self):
        with mock.patch('features.accounts.services.sign_in', return_value=STUDENT):
            response = self.client.post('/login/', {'email': 'est@example.com', 'password': 'clave-de-
prueba'})
        self.assertRedirects(response, '/', fetch_redirect_response=False)
        self.assertEqual(self.client.session['user'], STUDENT)

    def test_failed_login_shows_error_and_no_session(self):
        with mock.patch('features.accounts.services.sign_in', side_effect=AuthError('Correo o contraseña
incorrectos.')):
            response = self.client.post('/login/', {'email': 'est@example.com', 'password': 'x'})
        self.assertContains(response, 'Correo o contraseña incorrectos.')
        self.assertNotIn('user', self.client.session)

    def test_next_must_be_a_local_path(self):
        with mock.patch('features.accounts.services.sign_in', return_value=STUDENT):
            response = self.client.post('/login/?next=https://evil.example/', {'email': 'a@b.com', 'password':
'x'})
        self.assertRedirects(response, '/', fetch_redirect_response=False)

    def test_logout_clears_session_and_requires_post(self):
        self.login_as(STUDENT)
        self.assertEqual(self.client.get('/logout/').status_code, 405)
        self.client.post('/logout/')
        self.assertNotIn('user', self.client.session)


class CreateUserTests(SimpleTestCase):
    def test_register_student_always_uses_student_role(self):
        with mock.patch.object(services, 'create_user') as create:
            services.register_student('Ana', 'ana@example.com', 'abcdef')
        create.assert_called_once_with('Ana', 'ana@example.com', 'abcdef', role='student')

    def test_command_asks_password_and_creates_user(self):
        with mock.patch('features.accounts.management.commands.create_user.getpass', return_value='secreto1'),
\
                mock.patch.object(services, 'create_user') as create:
            call_command('create_user', 'bib@example.com', 'Bibliotecaria', '--role', 'librarian',
stdout=StringIO())
        create.assert_called_once_with('Bibliotecaria', 'bib@example.com', 'secreto1', 'librarian')

    def test_command_rejects_different_passwords(self):
        with mock.patch('features.accounts.management.commands.create_user.getpass', side_effect=['uno12345',
'dos12345']), \
                mock.patch.object(services, 'create_user') as create:
            with self.assertRaisesMessage(CommandError, 'no coinciden'):
                call_command('create_user', 'bib@example.com', 'Bibliotecaria')
        create.assert_not_called()
```

Ejecuta todas las pruebas:

TERMINAL

```
python manage.py test
```

SALIDA ESPERADA

```
Found 11 test(s).
System check identified no issues (0 silenced).
Creating test database for alias 'default'...
...........
```

```
----------------------------------------------------------------------
Ran 11 tests in 0.075s

OK
Destroying test database for alias 'default'...
```

✔ Verificación de la clase 1

- `python manage.py test` termina con **OK** (11 pruebas).
- Puedes ingresar, salir y registrarte en el navegador.
- En Firestore existe la colección `users` con el campo `role` correcto.

1.16 Errores comunes de la clase 1

Mensaje o síntoma	Causa probable	Solución
`command not found: django-admin` o `No module named django`	El entorno virtual no está activo	Ejecuta `source .venv/bin/activate` y mira que aparezca `(.venv)`
`ModuleNotFoundError: No module named 'dotenv'` (o `firebase_admin`)	Faltó instalar dependencias, o se instalaron fuera del entorno	Con el entorno activo: `pip install -r requirements.txt`
`FileNotFoundError ... serviceAccountKey.json`	La llave no está en `firebase/` o tiene otro nombre	Verifica la ruta `firebase/serviceAccountKey.json` y `FIREBASE_CREDENTIALS_PATH` en `.env`
`No se pudo iniciar sesión.` (mensaje genérico)	Firebase respondió con un error no traducido	Revisa `FIREBASE_WEB_API_KEY` en `.env`; confirma que activaste *Correo/contraseña* en Authentication
`NoReverseMatch: 'books' is not a registered namespace`	Una plantilla usa un enlace de una feature que aún no existe	Revisa que `base.html` sea exactamente el de este manual (los enlaces se agregan por clases)
`TemplateDoesNotExist: accounts/login.html`	El archivo está en otra carpeta o `features.accounts` no está en `INSTALLED_APPS`	Revisa la ruta `features/accounts/templates/accounts/login.html`
`Ya existe una cuenta con ese correo.`	Ese correo ya está registrado	Usa otro correo o bórralo en la consola de Firebase
`RuntimeError: Los tests no deben tocar Firestore`	Un test llamó a Firebase de verdad	Falta un `mock.patch` en ese test
Los cambios no aparecen en el navegador	Estilos en caché	Recarga forzada: **Ctrl + F5**

CLASE 2

Catálogo de libros, roles y datos de prueba

Duración estimada: 3 horas · **Al terminar:** el bibliotecario crea, edita y elimina libros; todos ven el catálogo con búsqueda y paginación; y tendrás 24 libros y 90 préstamos de prueba sembrados en Firestore.

> **Antes de empezar**
>
> Abre la terminal en la carpeta `biblioteca`, activa el entorno (`source .venv/bin/activate`, o el comando de Windows de la clase 1) y comprueba que `python manage.py test` termina en **OK**. Si no, repasa la clase 1 antes de seguir.

2.1 Qué vamos a hacer hoy

1. Agregar **paginación** reutilizable al módulo `core`.
2. Crear la feature `books`: servicios, formularios, vistas, rutas y plantillas.
3. Conectarla al proyecto (settings, rutas, menú) y reemplazar la página de inicio provisional.
4. Crear el **seeder**: un comando que siembra libros y préstamos de prueba.
5. Escribir las pruebas de la clase.

2.2 Roles: quién puede hacer qué

Ya guardamos el rol de cada usuario en Firestore. Ahora lo usamos para decidir qué puede hacer cada uno. Estas son las reglas de toda la aplicación (hoy implementamos las de libros):

Acción	Estudiante	Bibliotecario
Ver el catálogo y buscar	✔	✔
Crear, editar y eliminar libros	✘	✔
Prestar un libro (clase 3)	✔	✘
Ver y devolver **sus** préstamos (clase 3)	✔	✘
Ver, devolver o cancelar **cualquier** préstamo (clase 3)	✘	✔
Ver el dashboard (clase 4)	✘	✔

La protección se hace con los decoradores de `core/decorators.py` que ya escribimos. Y recuerda: ocultar un botón en la plantilla **no es seguridad**; la seguridad real es que la vista rechace la petición aunque alguien escriba la dirección a mano.

2.3 El modelo de datos de un libro

Cada libro será un **documento** de la colección `books`:

Campo	Tipo	Ejemplo
`title`	texto	`"Cien años de soledad"`
`author`	texto	`"Gabriel García Márquez"`
`category`	texto	`"Novela"`
`isbn`	texto	`"9780307474728"`
`copies_total`	número	`3`
`copies_available`	número	`2`

Fíjate: no guardamos "cuántos préstamos activos tiene", porque se puede **calcular**: `copies_total - copies_available`. Esa idea nos servirá para validar borrados y ediciones.

2.4 Paginación reutilizable en `core`

Con 24 libros (y más adelante 90 préstamos) una lista infinita es incómoda. Paginar es repartir los elementos en páginas de 10. Django ya trae un `Paginator`; nosotros lo envolvemos en **una función** que usarán todas las features.

`paginate(request, items)` devuelve un diccionario con tres cosas que la plantilla necesita: la página actual (`page_obj`), la lista de números a mostrar (`page_range`, con `…` cuando hay muchas) y el `querystring`, que conserva los filtros (`?q=python`) al cambiar de página.

core/pagination.py — ARCHIVO NUEVO

```python
"""Paginación sencilla de listas en memoria (los datos de este proyecto son pequeños)."""
from django.conf import settings
from django.core.paginator import Paginator


def paginate(request, items, per_page=None):
    """Devuelve el contexto de paginación para los templates.

    - page_obj: la página actual (sus elementos, número, si hay anterior/siguiente…)
    - page_range: números a mostrar, con "…" cuando hay muchas páginas
    - querystring: los filtros actuales (q, status…) sin `page`, para conservarlos en los enlaces
    """
    paginator = Paginator(items, per_page or settings.PAGE_SIZE)
    page_obj = paginator.get_page(request.GET.get('page'))
    params = request.GET.copy()
    params.pop('page', None)
    return {
        'page_obj': page_obj,
        'page_range': list(paginator.get_elided_page_range(page_obj.number, on_each_side=1, on_ends=1)),
        'querystring': params.urlencode(),
    }
```

Y su plantilla, un fragmento que cualquier lista puede incluir con `{% include 'core/_pagination.html' %}`:

core/templates/core/_pagination.html — ARCHIVO NUEVO

```
{% if page_obj.paginator.num_pages > 1 %}
<div class="pagination" aria-label="Paginación">
  {% if page_obj.has_previous %}
    <a role="button" class="outline secondary" href="?{% if querystring %}{{ querystring }}&{% endif %}page={{
page_obj.previous_page_number }}">‹ Anterior</a>
  {% else %}
    <span class="page-disabled">‹ Anterior</span>
  {% endif %}

  <span class="page-numbers">
    {% for p in page_range %}
      {% if p == page_obj.paginator.ELLIPSIS %}
        <span class="page-gap">…</span>
      {% elif p == page_obj.number %}
        <span class="page-current" aria-current="page">{{ p }}</span>
      {% else %}
        <a role="button" class="outline secondary" href="?{% if querystring %}{{ querystring }}&{% endif
%}page={{ p }}">{{ p }}</a>
      {% endif %}
    {% endfor %}
  </span>

  {% if page_obj.has_next %}
    <a role="button" class="outline secondary" href="?{% if querystring %}{{ querystring }}&{% endif %}page={{
page_obj.next_page_number }}">Siguiente ›</a>
  {% else %}
    <span class="page-disabled">Siguiente ›</span>
  {% endif %}
</div>
<p class="page-status">Página {{ page_obj.number }} de {{ page_obj.paginator.num_pages }} · {{
page_obj.paginator.count }} resultados</p>
{% endif %}
```

2.5 La feature `books`

Crea las carpetas y los `__init__.py`:

TERMINAL

```
mkdir -p features/books/templates/books
touch features/books/__init__.py
```

`apps.py`

features/books/apps.py — ARCHIVO NUEVO

```
from django.apps import AppConfig


class BooksConfig(AppConfig):
    name = 'features.books'
```

`services.py` · el catálogo en Firestore

Todo lo que toca la colección `books` está aquí, y **solo aquí**. Las vistas nunca hablan directo con Firestore.

Función	Qué hace
`books_collection()`	Devuelve la colección `books` (la usará `loans` en la clase 3)
`list_books(query, category)`	Lista ordenada por título, con filtro opcional de texto y categoría
`list_categories()`	Las categorías existentes (para el desplegable)
`get_book(book_id)`	Un libro por su identificador, o `None`
`create_book(...)`	Crea el libro con todas sus copias disponibles
`update_book(...)`	Edita el libro y **recalcula** las copias disponibles
`delete_book(book_id)`	Elimina el libro, solo si ninguna copia está prestada

Dos **reglas de negocio** viven aquí: no puedes bajar las copias totales por debajo de las que están prestadas, y no puedes eliminar un libro con copias prestadas. Ambas lanzan `BookError` con un mensaje listo para mostrar.

> **Por qué filtramos en Python**
>
> Firestore no puede buscar "títulos que contengan 'pyth'". Como nuestro catálogo es pequeño, traemos todos los libros y filtramos con Python (`list_books`). Para catálogos de miles de libros se usaría un buscador especializado.

features/books/services.py — ARCHIVO NUEVO

```python
"""Catálogo de libros (colección `books` en Firestore)."""
from core.firebase import get_db


class BookError(Exception):
    """Regla de negocio incumplida al modificar un libro."""


def books_collection():
    return get_db().collection('books')


def list_books(query='', category=''):
    """Lista libros ordenados por título; filtra por texto y categoría en Python."""
    books = [{'id': d.id, **d.to_dict()} for d in books_collection().stream()]
    query = query.strip().lower()
    if query:
        books = [b for b in books if query in b['title'].lower() or query in b['author'].lower()]
    if category:
        books = [b for b in books if b['category'] == category]
    return sorted(books, key=lambda b: b['title'].lower())


def list_categories():
    return sorted({d.to_dict()['category'] for d in books_collection().stream()})
```

```python
def get_book(book_id):
    snapshot = books_collection().document(book_id).get()
    return {'id': snapshot.id, **snapshot.to_dict()} if snapshot.exists else None


def create_book(title, author, category, isbn, copies):
    ref = books_collection().document()
    ref.set({
        'title': title, 'author': author, 'category': category, 'isbn': isbn,
        'copies_total': copies, 'copies_available': copies,
    })
    return ref.id


def update_book(book_id, title, author, category, isbn, copies):
    """Edita un libro; ajusta las copias disponibles según el nuevo total."""
    book = get_book(book_id)
    if book is None:
        raise BookError('El libro no existe.')
    on_loan = book['copies_total'] - book['copies_available']
    if copies < on_loan:
        raise BookError(f'No puedes bajar a {copies} copias: hay {on_loan} prestadas.')
    books_collection().document(book_id).update({
        'title': title, 'author': author, 'category': category, 'isbn': isbn,
        'copies_total': copies, 'copies_available': copies - on_loan,
    })


def delete_book(book_id):
    """Elimina un libro. Si hay copias prestadas (disponibles < totales) no se puede."""
    book = get_book(book_id)
    if book is None:
        raise BookError('El libro no existe.')
    if book['copies_available'] < book['copies_total']:
        raise BookError('No se puede eliminar: tiene préstamos activos.')
    books_collection().document(book_id).delete()
```

`forms.py`

El formulario define qué datos acepta un libro y sus límites (por ejemplo, entre 1 y 99 copias). Django valida y muestra los errores solo.

features/books/forms.py — ARCHIVO NUEVO

```python
from django import forms


class BookForm(forms.Form):
    title = forms.CharField(label='Título', max_length=150)
    author = forms.CharField(label='Autor', max_length=100)
    category = forms.CharField(label='Categoría', max_length=50)
    isbn = forms.CharField(label='ISBN', max_length=20, required=False)
    copies = forms.IntegerField(label='Copias totales', min_value=1, max_value=99, initial=1)
```

`views.py`

Cuatro vistas, una por acción. Observa cómo se apilan los decoradores en `book_delete`: primero el permiso (`@librarian_required`) y luego el método HTTP (`@require_POST`, porque borrar nunca debe hacerse con un enlace normal).

`book_list` es la más interesante: pide los libros al servicio, los pagina con `paginate()` y mezcla ese resultado con otros datos usando `**`. Así la plantilla recibe `page_obj`, `page_range`, `querystring`, `categories`, `q` y `category`.

features/books/views.py — ARCHIVO NUEVO

```
from django.contrib import messages
from django.shortcuts import redirect, render
from django.views.decorators.http import require_POST

from core.decorators import librarian_required, login_required
from core.pagination import paginate

from . import services
from .forms import BookForm
from .services import BookError


@login_required
def book_list(request):
    query = request.GET.get('q', '')
    category = request.GET.get('category', '')
    books = services.list_books(query, category)
    return render(request, 'books/list.html', {
        **paginate(request, books),
        'categories': services.list_categories(),
        'q': query,
        'category': category,
    })


@librarian_required
def book_create(request):
    form = BookForm(request.POST or None)
    if request.method == 'POST' and form.is_valid():
        d = form.cleaned_data
        services.create_book(d['title'], d['author'], d['category'], d['isbn'], d['copies'])
        messages.success(request, 'Libro creado.')
        return redirect('books:list')
    return render(request, 'books/form.html', {'form': form, 'title': 'Nuevo libro'})


@librarian_required
def book_edit(request, book_id):
    book = services.get_book(book_id)
    if book is None:
        messages.error(request, 'El libro no existe.')
        return redirect('books:list')
    form = BookForm(request.POST or None, initial={
        'title': book['title'], 'author': book['author'], 'category': book['category'],
        'isbn': book['isbn'], 'copies': book['copies_total'],
    })
    if request.method == 'POST' and form.is_valid():
        d = form.cleaned_data
        try:
            services.update_book(book_id, d['title'], d['author'], d['category'], d['isbn'], d['copies'])
        except BookError as exc:
            form.add_error('copies', str(exc))
        else:
            messages.success(request, 'Libro actualizado.')
            return redirect('books:list')
    return render(request, 'books/form.html', {'form': form, 'title': 'Editar libro'})
```

```python
@librarian_required
@require_POST
def book_delete(request, book_id):
    try:
        services.delete_book(book_id)
        messages.success(request, 'Libro eliminado.')
    except BookError as exc:
        messages.error(request, str(exc))
    return redirect('books:list')
```

`urls.py`

`<str:book_id>` captura el identificador del libro desde la dirección y lo entrega a la vista como argumento. El espacio de nombres es `books:`, de modo que en las plantillas usaremos `{% url 'books:edit' book.id %}`.

features/books/urls.py — ARCHIVO NUEVO

```python
from django.urls import path

from . import views

app_name = 'books'

urlpatterns = [
    path('', views.book_list, name='list'),
    path('new/', views.book_create, name='create'),
    path('<str:book_id>/edit/', views.book_edit, name='edit'),
    path('<str:book_id>/delete/', views.book_delete, name='delete'),
]
```

Plantillas de `books`

El formulario de crear y editar es el mismo, solo cambia el título:

features/books/templates/books/form.html — ARCHIVO NUEVO

```html
{% extends 'core/base.html' %}
{% block title %}{{ title }}{% endblock %}
{% block content %}
<article class="form-card">
  <h2>{{ title }}</h2>
  <form method="post">{% csrf_token %}
    {% include 'core/_form.html' %}
    <div class="actions">
      <button type="submit">Guardar</button>
      <a href="{% url 'books:list' %}" role="button" class="secondary outline">Cancelar</a>
    </div>
  </form>
</article>
{% endblock %}
```

La lista de libros. Puntos a notar:

- `{% for book in page_obj %}` recorre **solo la página actual**.
- `{% empty %}` se muestra cuando no hay resultados.
- `{% if current_user.role == ... %}` muestra distintos botones según el rol. El botón **Prestar** de los estudiantes lo agregaremos en la clase 3, por eso hoy hay un comentario `{# ... #}`.

- Las clases `table-cards` y los atributos `data-label` preparan la tabla para verse bien en el celular; los estilos llegan en la clase 4.

features/books/templates/books/list.html — ARCHIVO NUEVO

```
{% extends 'core/base.html' %}
{% block title %}Libros{% endblock %}
{% block content %}
<hgroup>
  <h1>Catálogo</h1>
  <p>{{ page_obj.paginator.count }} libro{{ page_obj.paginator.count|pluralize }}</p>
</hgroup>

<form method="get" class="search-form">
  <input type="search" name="q" value="{{ q }}" placeholder="Título o autor">
  <select name="category">
    <option value="">Todas las categorías</option>
    {% for c in categories %}<option value="{{ c }}" {% if c == category %}selected{% endif %}>{{ c }}
</option>{% endfor %}
  </select>
  <button type="submit">Buscar</button>
  {% if current_user.role == 'librarian' %}
    <a href="{% url 'books:create' %}" role="button" class="secondary">+ Nuevo libro</a>
  {% endif %}
</form>

<figure>
<table class="table-cards books-table">
  <thead><tr><th>Título</th><th>Autor</th><th>Categoría</th><th>Disponibles</th><th></th></tr></thead>
  <tbody>
  {% for book in page_obj %}
    <tr>
      <td data-label="Título">{{ book.title }}</td>
      <td data-label="Autor">{{ book.author }}</td>
      <td data-label="Categoría">{{ book.category }}</td>
      <td data-label="Disponibles">{{ book.copies_available }} / {{ book.copies_total }}</td>
      <td class="cell-actions">
        <div class="actions">
        {% if current_user.role == 'student' %}
          {# Clase 3: aquí irá el botón "Prestar" #}
        {% else %}
          <a href="{% url 'books:edit' book.id %}" role="button" class="outline">Editar</a>
          <form method="post" action="{% url 'books:delete' book.id %}" onsubmit="return confirm('¿Eliminar
este libro?')">{% csrf_token %}
            <button type="submit" class="outline secondary">Eliminar</button>
          </form>
        {% endif %}
        </div>
      </td>
    </tr>
  {% empty %}
    <tr><td colspan="5">No hay libros que coincidan.</td></tr>
  {% endfor %}
  </tbody>
</table>
</figure>
{% include 'core/_pagination.html' %}
{% endblock %}
```

Estilos de la clase 2

Agrega al **final** de `core/static/core/css/site.css` tres secciones nuevas: el buscador, los botones de acción de las tablas y la paginación.

core/static/core/css/site.css — AGREGA AL FINAL

```css
/* --- Buscador y filtros ------------------------------------------------ */
.search-form { display: flex; flex-wrap: wrap; gap: .5rem; align-items: center; margin-bottom: 1.25rem; }
.search-form :is(input, select, button, a) { margin: 0; width: auto; }
.search-form input[type=search] { flex: 1 1 14rem; min-width: 0; }
.search-form select { flex: 0 1 14rem; min-width: 0; }
.search-form button, .search-form a[role=button] { white-space: nowrap; }
.filter-form { max-width: 18rem; }

/* --- Acciones en tablas ------------------------------------------------ */
td form { margin: 0; }
.actions { display: flex; gap: .5rem; flex-wrap: wrap; }
.books-table td .actions { flex-wrap: nowrap; }  /* en escritorio Editar/Eliminar van en una línea */
.actions button, .actions a { margin: 0; padding: .25rem .7rem; font-size: .85rem; width: auto; }

/* --- Paginación -------------------------------------------------------- */
.pagination { display: flex; flex-wrap: wrap; gap: .4rem; justify-content: center; align-items: center;
margin-top: 1.5rem; }
.pagination a, .pagination span.page-current, .pagination span.page-disabled {
  margin: 0; width: auto; padding: .3rem .75rem; font-size: .9rem; border-radius: var(--pico-border-radius);
}
.page-numbers { display: flex; flex-wrap: wrap; gap: .4rem; align-items: center; }
.page-current { background: var(--pico-primary-background); color: var(--pico-primary-inverse); font-weight:
600; }
.page-disabled, .page-gap { color: var(--pico-muted-color); }
.page-status { margin-top: .6rem; text-align: center; font-size: .85rem; color: var(--pico-muted-color); }
@media (max-width: 480px) { .page-numbers { display: none; } }  /* en móvil solo Anterior / Siguiente */
```

2.6 Conectar `books` con el proyecto

Cuatro cambios pequeños.

1. Registrar la app en `config/settings.py`:

MODIFICA `config/settings.py`

BUSCA este texto (está tal cual en el archivo):

```
    'features.accounts',
]
```

y REEMPLÁZALO por:

```
    'features.accounts',
    'features.books',
]
```

2. Agregar la ruta en `config/urls.py`. El prefijo `books/` hace que todas las direcciones de la feature empiecen por `/books/`:

MODIFICA `config/urls.py`

BUSCA este texto (está tal cual en el archivo):

```
    path('', include('features.accounts.urls')),
]
```

y REEMPLÁZALO por:

```
    path('', include('features.accounts.urls')),
    path('books/', include('features.books.urls')),
]
```

3. Agregar el enlace *Libros* al menú en `core/templates/core/base.html`:

MODIFICA `core/templates/core/base.html`

BUSCA este texto (está tal cual en el archivo):

```
        {% if current_user %}
```

y REEMPLÁZALO por:

```
        {% if current_user %}
          <li><a href="{% url 'books:list' %}">Libros</a></li>
```

4. Reemplazar la página de inicio. Ahora que existe el catálogo, la página `/` ya no necesita mostrar un saludo provisional: por ahora lleva a todos al catálogo. (En la clase 4 haremos que el bibliotecario aterrice en el dashboard.) **Reemplaza todo** `core/views.py`:

core/views.py — REEMPLAZA TODO EL ARCHIVO

```
from django.shortcuts import redirect


def home(request):
    """Por ahora todos aterrizan en el catálogo; en la clase 4 el bibliotecario irá al dashboard."""
    return redirect('books:list')
```

La página provisional ya no se usa; bórrala:

TERMINAL

```
rm core/templates/core/home.html
```

2.7 Probar el catálogo

TERMINAL

```
python manage.py runserver
```

1. Ingresa con tu **bibliotecario** y abre http://127.0.0.1:8000/books/. Verás el catálogo vacío.
2. Pulsa **+ Nuevo libro**, llena el formulario y guarda. El libro aparece en la lista.

3. Pulsa **Editar**, cambia las copias y guarda. Prueba poner un número menor a las copias prestadas (todavía no hay ninguna).
4. En la consola de Firebase, abre **Firestore → Datos**: debe existir la colección `books` con tu documento.
5. Cierra sesión e ingresa con un **estudiante**: ve el catálogo, pero sin botones de edición.



Formulario para crear un libro

2.8 El seeder: datos de prueba

Para probar la paginación, las gráficas y los préstamos necesitamos **datos**, y escribirlos a mano sería eterno. Un **seeder** (*sembrador*) es un script que crea datos de ejemplo. El nuestro es un comando de Django: `python manage.py seed`.

Qué hace, en orden:

1. Lee los **estudiantes** que existen en Firestore (los que creaste en la clase 1). Si no hay ninguno, se detiene con un mensaje.
2. **Borra** los libros y préstamos actuales (los usuarios no se tocan).
3. Crea **24 libros** de varias categorías con 2 a 5 copias.
4. Genera **90 préstamos** repartidos en los últimos 6 meses: los viejos casi siempre están devueltos, algunos cancelados, y los recientes suelen estar activos. Los más atrasados quedan **vencidos**.
5. Respeta las reglas: nunca hay más préstamos activos que copias, y ningún estudiante pasa de 3 activos. Al final calcula `copies_available` de cada libro.
6. Escribe todo en Firestore con un **lote** (`batch`), que guarda varios documentos en una sola operación.

Usa `random.seed(42)`, así que cada ejecución genera **los mismos** datos. Eso es útil para que tus resultados se parezcan a los de esta guía.

TERMINAL

```
mkdir -p features/seeder/management/commands
touch features/seeder/__init__.py features/seeder/management/__init__.py
features/seeder/management/commands/__init__.py
```

features/seeder/apps.py — ARCHIVO NUEVO

```python
from django.apps import AppConfig


class SeederConfig(AppConfig):
    name = 'features.seeder'
```

features/seeder/management/commands/seed.py — ARCHIVO NUEVO

```python
"""Siembra libros y préstamos de ejemplo en Firestore.

Uso:  python manage.py seed
Borra los libros y préstamos actuales y los vuelve a crear. Los usuarios NO se
tocan: los préstamos se reparten entre los estudiantes que ya existan.
"""
import random
from collections import Counter
from datetime import timedelta

from django.conf import settings
from django.core.management.base import BaseCommand, CommandError
from django.utils import timezone

from core.firebase import get_db
from features.accounts import services as accounts_service

BOOKS = [
    ('Cien años de soledad', 'Gabriel García Márquez', 'Novela', '9780307474728'),
    ('El coronel no tiene quien le escriba', 'Gabriel García Márquez', 'Novela', '9780060732714'),
    ('La vorágine', 'José Eustasio Rivera', 'Novela', '9788437604947'),
    ('Rayuela', 'Julio Cortázar', 'Novela', '9788437604572'),
    ('Pedro Páramo', 'Juan Rulfo', 'Novela', '9788437604947'),
    ('Don Quijote de la Mancha', 'Miguel de Cervantes', 'Clásicos', '9788420412146'),
    ('La Odisea', 'Homero', 'Clásicos', '9788424936518'),
    ('Crimen y castigo', 'Fiódor Dostoievski', 'Clásicos', '9788491050483'),
    ('Python Crash Course', 'Eric Matthes', 'Programación', '9781718502703'),
    ('Fluent Python', 'Luciano Ramalho', 'Programación', '9781492056355'),
    ('Clean Code', 'Robert C. Martin', 'Programación', '9780132350884'),
    ('Django for Beginners', 'William S. Vincent', 'Programación', '9781735467207'),
    ('Introduction to Algorithms', 'Thomas H. Cormen', 'Programación', '9780262046305'),
    ('Una breve historia del tiempo', 'Stephen Hawking', 'Ciencia', '9788498929294'),
    ('Cosmos', 'Carl Sagan', 'Ciencia', '9788408175216'),
    ('El gen egoísta', 'Richard Dawkins', 'Ciencia', '9788434413443'),
    ('Sapiens', 'Yuval Noah Harari', 'Historia', '9788499926223'),
    ('Armas, gérmenes y acero', 'Jared Diamond', 'Historia', '9788483068014'),
    ('Breve historia de Colombia', 'Jorge Orlando Melo', 'Historia', '9789587143393'),
    ('El principito', 'Antoine de Saint-Exupéry', 'Infantil', '9788478887194'),
    ('Matilda', 'Roald Dahl', 'Infantil', '9788420482514'),
    ('Padre rico, padre pobre', 'Robert Kiyosaki', 'Economía', '9781612680194'),
    ('Pensar rápido, pensar despacio', 'Daniel Kahneman', 'Economía', '9788499923260'),
    ('Hábitos atómicos', 'James Clear', 'Desarrollo personal', '9786075694566'),
]

MONTHS_BACK = 6          # los préstamos se reparten en los últimos 6 meses
```

```python
LOANS_TO_CREATE = 90


class Command(BaseCommand):
    help = 'Crea libros y préstamos de ejemplo en Firestore.'

    def add_arguments(self, parser):
        parser.add_argument('--seed', type=int, default=42, help='Semilla aleatoria (resultados repetibles).')

    def handle(self, *args, **options):
        random.seed(options['seed'])
        db = get_db()

        students = accounts_service.list_students()
        if not students:
            raise CommandError('No hay estudiantes en Firestore. Crea o registra al menos uno primero.')

        self._clear(db, 'loans')
        self._clear(db, 'books')

        books = self._create_books(db)
        loans = self._generate_loans(books, students)
        self._write(db, books, loans)

        statuses = Counter(loan['status'] for loan in loans)
        self.stdout.write(self.style.SUCCESS(
            f'Listo: {len(books)} libros y {len(loans)} préstamos '
            f'({statuses["active"]} activos, {statuses["returned"]} devueltos, '
            f'{statuses["cancelled"]} cancelados) para {len(students)} estudiantes.'
        ))

    def _clear(self, db, collection):
        for doc in db.collection(collection).stream():
            doc.reference.delete()

    def _create_books(self, db):
        books = []
        for title, author, category, isbn in BOOKS:
            copies = random.randint(2, 5)
            ref = db.collection('books').document()
            books.append({
                'ref': ref, 'title': title, 'author': author, 'category': category,
                'isbn': isbn, 'copies_total': copies,
            })
        return books

    def _generate_loans(self, books, students):
        """Genera el historial en memoria respetando stock y límite por estudiante."""
        now = timezone.now()
        # Libros populares: algunos pesan más para que el "top" sea interesante
        weights = [random.choice([1, 1, 2, 3, 6]) for _ in books]
        loans = []
        active_by_book, active_by_student = Counter(), Counter()

        for _ in range(LOANS_TO_CREATE):
            book = random.choices(books, weights)[0]
            student = random.choice(students)
            age_days = random.randint(0, MONTHS_BACK * 30)
            loaned_at = now - timedelta(days=age_days, hours=random.randint(0, 10))
            due_at = loaned_at + timedelta(days=settings.LOAN_DAYS)

            roll = random.random()
            if age_days < settings.LOAN_DAYS + 7:
                status = 'active' if roll < 0.6 else 'returned'
            else:
                status = 'returned' if roll < 0.85 else ('cancelled' if roll < 0.95 else 'active')
```

```
            if status == 'active' and (
                active_by_book[book['ref'].id] >= book['copies_total']
                or active_by_student[student['uid']] >= settings.MAX_ACTIVE_LOANS
            ):
                status = 'returned'

            returned_at = cancelled_at = None
            if status == 'active':
                active_by_book[book['ref'].id] += 1
                active_by_student[student['uid']] += 1
            elif status == 'returned':
                returned_at = min(loaned_at + timedelta(days=random.randint(1, settings.LOAN_DAYS + 6)), now)
            else:
                cancelled_at = min(loaned_at + timedelta(hours=random.randint(1, 36)), now)

            loans.append({
                'book_id': book['ref'].id, 'book_title': book['title'], 'book_category': book['category'],
                'user_id': student['uid'], 'user_name': student['name'], 'user_email': student['email'],
                'status': status, 'loaned_at': loaned_at, 'due_at': due_at,
                'returned_at': returned_at, 'cancelled_at': cancelled_at,
            })

        for book in books:
            book['copies_available'] = book['copies_total'] - active_by_book[book['ref'].id]
        return loans

    def _write(self, db, books, loans):
        batch = db.batch()
        for book in books:
            data = {k: v for k, v in book.items() if k != 'ref'}
            batch.set(book['ref'], data)
        for loan in loans:
            batch.set(db.collection('loans').document(), loan)
        batch.commit()
```

Registra la app en `config/settings.py`, **al final** de la lista (después de `features.books`):

MODIFICA `config/settings.py`

BUSCA este texto (está tal cual en el archivo):

```
    'features.books',
]
```

y REEMPLÁZALO por:

```
    'features.books',
    'features.seeder',
]
```

Siembra los datos:

TERMINAL

```
python manage.py seed
```

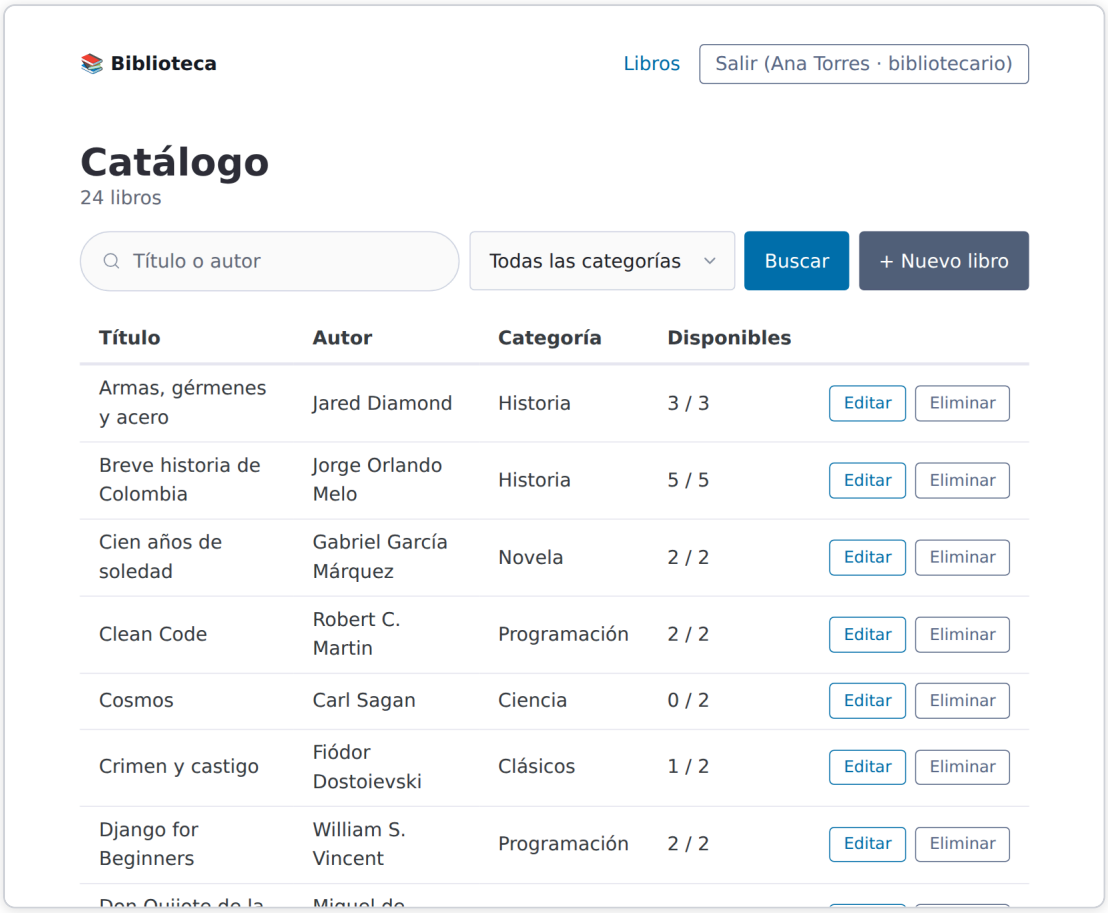
SALIDA ESPERADA

```
Listo: 24 libros y 90 préstamos (5 activos, 78 devueltos, 7 cancelados) para 2 estudiantes.
```

El seeder borra

`seed` elimina **todos** los libros y préstamos antes de crear los nuevos. Úsalo en tu base de pruebas, no cuando ya tengas datos reales. Puedes ejecutarlo cuantas veces quieras para volver a un estado conocido.

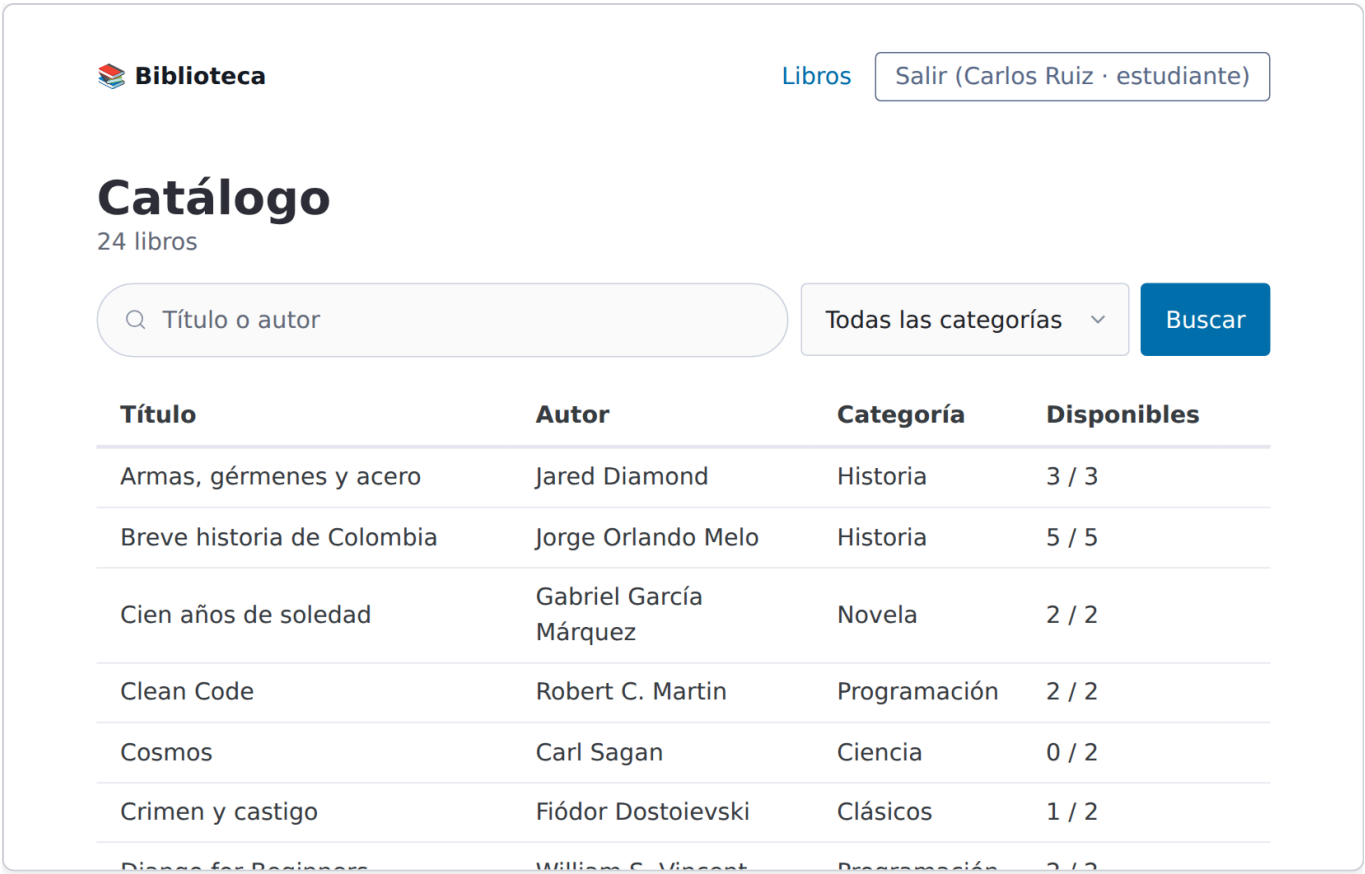
Recarga http://127.0.0.1:8000/books/: ahora hay 24 libros repartidos en 3 páginas.



Catálogo visto por el bibliotecario



Paginación al final de la lista



Catálogo visto por un estudiante: sin botones de edición

Prueba la búsqueda (`python` , `gabriel`), el filtro por categoría y fíjate en que, al cambiar de página, **la búsqueda se conserva** en la dirección (`?q=python&page=2`). Eso es el `querystring` que calcula `paginate()` .

2.9 Tests de la clase 2

Tres archivos de pruebas, todos sin tocar Firebase.

`core/tests.py` prueba la paginación: que reparte bien, que una página inválida no rompe nada y que con muchas páginas aparece el `…` :

core/tests.py — ARCHIVO NUEVO

```
from django.core.paginator import Paginator
from django.test import RequestFactory, SimpleTestCase

from .pagination import paginate


class PaginateTests(SimpleTestCase):
    def test_splits_items_and_keeps_filters(self):
        request = RequestFactory().get('/books/', {'q': 'python', 'page': '2'})
        ctx = paginate(request, list(range(25)), per_page=10)
        self.assertEqual(list(ctx['page_obj']), list(range(10, 20)))
        self.assertEqual(ctx['page_obj'].paginator.num_pages, 3)
        self.assertEqual(ctx['querystring'], 'q=python')  # sin `page`

    def test_invalid_page_falls_back_to_valid_one(self):
        request = RequestFactory().get('/', {'page': '999'})
        self.assertEqual(paginate(request, list(range(25)), per_page=10)['page_obj'].number, 3)
```

```
        request = RequestFactory().get('/', {'page': 'abc'})
        self.assertEqual(paginate(request, list(range(25)), per_page=10)['page_obj'].number, 1)

    def test_many_pages_are_elided(self):
        request = RequestFactory().get('/', {'page': '10'})
        ctx = paginate(request, list(range(200)), per_page=10)
        self.assertIn(Paginator.ELLIPSIS, ctx['page_range'])
        self.assertLess(len(ctx['page_range']), 20)
```

`features/books/tests.py` prueba el formulario, las dos reglas de negocio del servicio y las vistas. Mira cómo `mock.patch.object(services, 'list_books', ...)` reemplaza la función real por una lista inventada: así probamos la **vista** sin Firestore.

features/books/tests.py — ARCHIVO NUEVO

```
from unittest import mock

from django.test import SimpleTestCase

from core.testing import LIBRARIAN, STUDENT, SessionTestCase

from . import services
from .forms import BookForm
from .services import BookError


def make_books(n):
    return [
        {'id': f'b{i}', 'title': f'Libro {i:02}', 'author': 'Autor', 'category': 'Novela',
         'isbn': '', 'copies_total': 2, 'copies_available': 2}
        for i in range(n)
    ]


class BookFormTests(SimpleTestCase):
    def test_needs_at_least_one_copy(self):
        form = BookForm({'title': 'X', 'author': 'Y', 'category': 'Z', 'copies': 0})
        self.assertFalse(form.is_valid())
        self.assertIn('copies', form.errors)

    def test_isbn_is_optional(self):
        self.assertTrue(BookForm({'title': 'X', 'author': 'Y', 'category': 'Z', 'copies': 1}).is_valid())


class BookServiceRulesTests(SimpleTestCase):
    def book(self, total, available):
        return {'id': 'b1', 'copies_total': total, 'copies_available': available}

    def test_cannot_reduce_copies_below_those_on_loan(self):
        with mock.patch.object(services, 'get_book', return_value=self.book(5, 2)):  # 3 prestadas
            with self.assertRaises(BookError):
                services.update_book('b1', 'T', 'A', 'C', '', copies=2)

    def test_cannot_delete_book_with_copies_on_loan(self):
        with mock.patch.object(services, 'get_book', return_value=self.book(3, 2)):
            with self.assertRaisesMessage(BookError, 'préstamos activos'):
                services.delete_book('b1')

    def test_cannot_delete_missing_book(self):
        with mock.patch.object(services, 'get_book', return_value=None):
            with self.assertRaises(BookError):
                services.delete_book('nope')
```

```
class BookViewTests(SessionTestCase):
    def test_list_is_paginated(self):
        self.login_as(STUDENT)
        with mock.patch.object(services, 'list_books', return_value=make_books(25)), \
                mock.patch.object(services, 'list_categories', return_value=['Novela']):
            page1 = self.client.get('/books/')
            page3 = self.client.get('/books/?page=3')
        self.assertContains(page1, 'Libro 09')
        self.assertNotContains(page1, 'Libro 10')
        self.assertContains(page3, 'Libro 24')
        self.assertContains(page1, '25 libros')

    def test_pagination_links_keep_the_filters(self):
        self.login_as(STUDENT)
        with mock.patch.object(services, 'list_books', return_value=make_books(25)), \
                mock.patch.object(services, 'list_categories', return_value=[]):
            response = self.client.get('/books/?q=libro')
        self.assertContains(response, '?q=libro&page=2')

    def test_student_cannot_create_or_delete_books(self):
        self.login_as(STUDENT)
        self.assertRedirects(self.client.get('/books/new/'), '/books/', fetch_redirect_response=False)
        with mock.patch.object(services, 'delete_book') as delete:
            self.client.post('/books/b1/delete/')
        delete.assert_not_called()

    def test_librarian_creates_book(self):
        self.login_as(LIBRARIAN)
        with mock.patch.object(services, 'create_book') as create:
            response = self.client.post('/books/new/', {
                'title': 'Nuevo', 'author': 'Autor', 'category': 'Ciencia', 'isbn': '', 'copies': 3})
        self.assertRedirects(response, '/books/', fetch_redirect_response=False)
        create.assert_called_once_with('Nuevo', 'Autor', 'Ciencia', '', 3)

    def test_delete_error_is_shown_as_message(self):
        self.login_as(LIBRARIAN)
        with mock.patch.object(services, 'delete_book', side_effect=BookError('No se puede eliminar')), \
                mock.patch.object(services, 'list_books', return_value=[]), \
                mock.patch.object(services, 'list_categories', return_value=[]):
            response = self.client.post('/books/b1/delete/', follow=True)
        self.assertContains(response, 'No se puede eliminar')
```

`features/seeder/tests.py` no escribe nada: genera los préstamos **en memoria** y comprueba sus invariantes (nunca más activos que copias, máximo 3 por estudiante, fechas coherentes):

features/seeder/tests.py — ARCHIVO NUEVO

```
import random
from collections import Counter
from datetime import timedelta
from types import SimpleNamespace

from django.conf import settings
from django.test import SimpleTestCase

from .management.commands.seed import BOOKS, Command


class SeedGenerationTests(SimpleTestCase):
    """Comprueba los invariantes de los datos generados, sin escribir en Firestore."""

    def setUp(self):
```

```
        random.seed(7)
        self.books = [
            {'ref': SimpleNamespace(id=f'b{i}'), 'title': t, 'author': a, 'category': c, 'isbn': i_,
             'copies_total': random.randint(2, 5)}
            for i, (t, a, c, i_) in enumerate(BOOKS)
        ]
        self.students = [{'uid': f's{i}', 'name': f'Est {i}', 'email': f'e{i}@x.com'} for i in range(2)]
        self.loans = Command()._generate_loans(self.books, self.students)

    def test_never_more_active_loans_than_copies(self):
        active = Counter(loan['book_id'] for loan in self.loans if loan['status'] == 'active')
        for book in self.books:
            self.assertLessEqual(active[book['ref'].id], book['copies_total'])

    def test_available_copies_match_active_loans(self):
        active = Counter(loan['book_id'] for loan in self.loans if loan['status'] == 'active')
        for book in self.books:
            self.assertEqual(book['copies_available'], book['copies_total'] - active[book['ref'].id])

    def test_students_respect_the_active_loan_limit(self):
        active = Counter(loan['user_id'] for loan in self.loans if loan['status'] == 'active')
        self.assertTrue(all(n <= settings.MAX_ACTIVE_LOANS for n in active.values()))

    def test_dates_are_coherent(self):
        for loan in self.loans:
            self.assertEqual(loan['due_at'] - loan['loaned_at'], timedelta(days=settings.LOAN_DAYS))
            if loan['returned_at']:
                self.assertGreaterEqual(loan['returned_at'], loan['loaned_at'])
            if loan['cancelled_at']:
                self.assertGreaterEqual(loan['cancelled_at'], loan['loaned_at'])

    def test_has_a_mix_of_statuses(self):
        self.assertGreaterEqual(len({loan['status'] for loan in self.loans}), 3)
```

TERMINAL

```
python manage.py test
```

SALIDA ESPERADA

```
Found 29 test(s).
System check identified no issues (0 silenced).
Creating test database for alias 'default'...
.............................
----------------------------------------------------------------------
Ran 29 tests in 0.135s

OK
Destroying test database for alias 'default'...
```

✔ Verificación de la clase 2

- `python manage.py test` termina con **OK** (29 pruebas).
- El bibliotecario crea, edita y elimina libros; el estudiante solo los ve.
- El catálogo muestra 24 libros en 3 páginas y la búsqueda se conserva al paginar.
- En Firestore existen las colecciones `books` y `loans`.

2.10 Errores comunes de la clase 2

Mensaje o síntoma	Causa probable	Solución
`ModuleNotFoundError: No module named 'features.books'`	Falta `features/books/__init__.py` o `INSTALLED_APPS`	Crea el archivo vacío y revisa `'features.books'` en `settings.py`
`NoReverseMatch: Reverse for 'create' not found`	Falta el `app_name = 'books'` en `features/books/urls.py`	Revisa que esa línea exista
`TemplateDoesNotExist: books/list.html`	La plantilla está en una carpeta equivocada	Debe estar en `features/books/templates/books/list.html` (el nombre `books` se repite dos veces)
`No hay estudiantes en Firestore...` al sembrar	No creaste estudiantes	Usa `python manage.py create_user ...` (clase 1)
El buscador no filtra	El campo del formulario no se llama `q`	Compara con `books/list.html`
`RuntimeError: Los tests no deben tocar Firestore`	Un test llama a un servicio sin `mock.patch`	Parchea la función que usa la vista
La paginación no aparece	Hay 10 libros o menos	Es correcto: solo se muestra con más de una página

CLASE 3

Préstamos: prestar, devolver y cancelar

Duración estimada: 3 horas · **Al terminar:** los estudiantes prestan libros y gestionan sus préstamos; el bibliotecario ve y administra todos; y el stock de copias nunca se descuadra.

> **Antes de empezar**
>
> Activa el entorno virtual y comprueba que `python manage.py test` termina en **OK** (29 pruebas) y que tienes datos sembrados (`python manage.py seed`). Esta es la clase con más lógica: tómate el tiempo de leer las explicaciones.

3.1 El flujo de un préstamo

Un préstamo es un documento de la colección `loans` que nace **activo** y termina de dos formas:

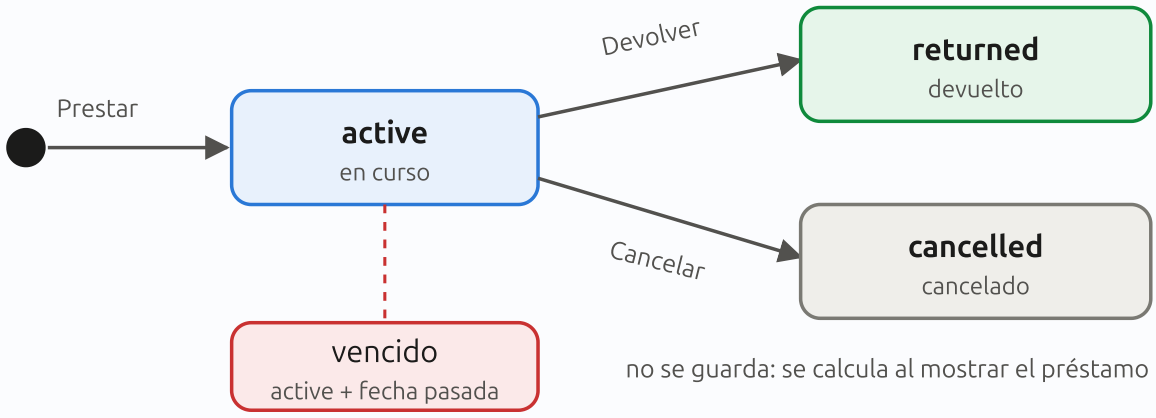


Figura: estados de un préstamo

- **Devolver:** el estudiante entrega el libro. El préstamo pasa a `returned` y la copia vuelve al catálogo.
- **Cancelar:** el préstamo se hizo por error y se anula. Pasa a `cancelled` y la copia también vuelve.
- **Vencido** no es un estado guardado: es un préstamo `active` cuya fecha límite (`due_at`) ya pasó. Se **calcula** al mostrarlo.

Estas son las **reglas de negocio**:

Regla	Mensaje cuando se incumple
El libro debe existir	*El libro no existe.*
Debe haber al menos una copia disponible	*No hay copias disponibles de este libro.*
Máximo 3 préstamos activos por estudiante (`MAX_ACTIVE_LOANS`)	*Ya tienes 3 préstamos activos.*

Regla	Mensaje cuando se incumple
No se puede prestar el mismo libro dos veces a la misma persona	*Ya tienes un préstamo activo de este libro.*
El plazo es de 14 días (`LOAN_DAYS`)	(se calcula solo)
Solo el dueño o un bibliotecario puede devolver o cancelar	*Ese préstamo no es tuyo.*
Solo se puede cerrar un préstamo que sigue activo	*Ese préstamo ya no está activo.*

Cada préstamo guarda una **copia** del título, la categoría y el nombre del estudiante (*desnormalización*). Así, la lista de préstamos se muestra sin consultar otras colecciones:

Campo	Ejemplo
`book_id` , `book_title` , `book_category`	`"a1b2c3"` , `"Clean Code"` , `"Programación"`
`user_id` , `user_name` , `user_email`	`"uid-del-estudiante"` , `"Laura Gómez"` , `"laura@..."`
`status`	`"active"` · `"returned"` · `"cancelled"`
`loaned_at` , `due_at`	fecha de préstamo y fecha límite (+14 días)
`returned_at` , `cancelled_at`	`null` hasta que ocurre

3.2 La feature `loans`

TERMINAL

```
mkdir -p features/loans/templates/loans
touch features/loans/__init__.py
```

features/loans/apps.py — ARCHIVO NUEVO

```python
from django.apps import AppConfig


class LoansConfig(AppConfig):
    name = 'features.loans'
```

3.3 `rules.py` : las reglas como funciones puras

Una **función pura** recibe datos y devuelve un resultado (o lanza un error) **sin tocar ninguna base de datos**. Poner las reglas en funciones puras tiene una gran ventaja: se pueden probar en milisegundos, sin Firebase ni internet.

Fíjate en la forma: `check_can_borrow(book, book_id, active_loans)` no consulta nada; **recibe** el libro y los préstamos activos ya leídos, y decide si el préstamo es válido. Más adelante, el servicio los leerá de Firestore y se los pasará.

features/loans/rules.py — ARCHIVO NUEVO

```python
"""Reglas de negocio de los préstamos como funciones puras (sin Firestore).

Los servicios leen los datos dentro de una transacción y llaman a estas funciones
para decidir si la operación es válida. Así las reglas se pueden probar sin base de datos.
"""
from django.conf import settings

ACTIVE, RETURNED, CANCELLED = 'active', 'returned', 'cancelled'


class LoanError(Exception):
    """Regla de negocio incumplida; el mensaje se muestra al usuario."""


def check_can_borrow(book, book_id, active_loans):
    """`book`: dict del libro (o None). `active_loans`: préstamos activos del estudiante."""
    if book is None:
        raise LoanError('El libro no existe.')
    if book['copies_available'] < 1:
        raise LoanError('No hay copias disponibles de este libro.')
    if len(active_loans) >= settings.MAX_ACTIVE_LOANS:
        raise LoanError(f'Ya tienes {settings.MAX_ACTIVE_LOANS} préstamos activos.')
    if any(loan['book_id'] == book_id for loan in active_loans):
        raise LoanError('Ya tienes un préstamo activo de este libro.')


def check_can_close(loan, user):
    """Devolver o cancelar: solo el dueño o un bibliotecario, y solo si sigue activo."""
    if loan is None:
        raise LoanError('El préstamo no existe.')
    if user['role'] != 'librarian' and loan['user_id'] != user['uid']:
        raise LoanError('Ese préstamo no es tuyo.')
    if loan['status'] != ACTIVE:
        raise LoanError('Ese préstamo ya no está activo.')


def is_overdue(loan, now):
    return loan['status'] == ACTIVE and loan['due_at'] < now
```

Primero las pruebas de las reglas

Probamos las reglas **antes** de construir el resto. Cada prueba describe una regla de la tabla anterior. Observa los ayudantes `loan(**overrides)` (crea un préstamo de ejemplo) y la constante `NOW` (una fecha fija, para que las pruebas no dependan del día en que se ejecutan).

features/loans/tests.py — ARCHIVO NUEVO

```python
from datetime import datetime, timedelta, timezone as dt_timezone
from unittest import mock

from django.test import SimpleTestCase

from core.testing import LIBRARIAN, STUDENT, SessionTestCase

from . import rules
from .rules import LoanError

NOW = datetime(2026, 10, 16, 12, 0, tzinfo=dt_timezone.utc)
```

```
def loan(**overrides):
    data = {
        'id': 'l1', 'book_id': 'b1', 'book_title': 'Clean Code', 'book_category': 'Programación',
        'user_id': STUDENT['uid'], 'user_name': 'Estudiante', 'user_email': 'est@example.com',
        'status': 'active', 'loaned_at': NOW - timedelta(days=3), 'due_at': NOW + timedelta(days=11),
        'overdue': False,
    }
    return {**data, **overrides}


class BorrowRulesTests(SimpleTestCase):
    book = {'title': 'Clean Code', 'copies_available': 1}

    def test_valid_borrow(self):
        rules.check_can_borrow(self.book, 'b1', active_loans=[])

    def test_book_must_exist(self):
        with self.assertRaisesMessage(LoanError, 'no existe'):
            rules.check_can_borrow(None, 'b1', [])

    def test_needs_available_copies(self):
        with self.assertRaisesMessage(LoanError, 'No hay copias'):
            rules.check_can_borrow({'copies_available': 0}, 'b1', [])

    def test_limit_of_active_loans(self):
        active = [loan(book_id=f'x{i}') for i in range(3)]
        with self.assertRaisesMessage(LoanError, 'Ya tienes 3 préstamos activos'):
            rules.check_can_borrow(self.book, 'b1', active)

    def test_cannot_borrow_same_book_twice(self):
        with self.assertRaisesMessage(LoanError, 'Ya tienes un préstamo activo de este libro'):
            rules.check_can_borrow(self.book, 'b1', [loan(book_id='b1')])

    def test_can_borrow_different_book_with_one_active(self):
        rules.check_can_borrow(self.book, 'b1', [loan(book_id='b2')])


class CloseRulesTests(SimpleTestCase):
    def test_owner_can_close(self):
        rules.check_can_close(loan(), STUDENT)

    def test_librarian_can_close_anyones_loan(self):
        rules.check_can_close(loan(), LIBRARIAN)

    def test_other_student_cannot(self):
        other = {**STUDENT, 'uid': 'otro'}
        with self.assertRaisesMessage(LoanError, 'no es tuyo'):
            rules.check_can_close(loan(), other)

    def test_only_active_loans_can_be_closed(self):
        for status in ('returned', 'cancelled'):
            with self.assertRaisesMessage(LoanError, 'ya no está activo'):
                rules.check_can_close(loan(status=status), STUDENT)

    def test_missing_loan(self):
        with self.assertRaises(LoanError):
            rules.check_can_close(None, STUDENT)


class OverdueTests(SimpleTestCase):
    def test_active_past_due_is_overdue(self):
        self.assertTrue(rules.is_overdue(loan(due_at=NOW - timedelta(days=1)), NOW))

    def test_active_before_due_is_not(self):
        self.assertFalse(rules.is_overdue(loan(due_at=NOW + timedelta(days=1)), NOW))
```

```
    def test_returned_is_never_overdue(self):
        self.assertFalse(rules.is_overdue(loan(status='returned', due_at=NOW - timedelta(days=9)), NOW))
```

TERMINAL

```
python manage.py test features.loans
```

SALIDA ESPERADA

```
Found 14 test(s).
System check identified no issues (0 silenced).
..............
----------------------------------------------------------------------
Ran 14 tests in 0.002s

OK
```

> ✔ **Verificación**
>
> Deben pasar las **14 pruebas** de reglas. Si alguna falla, el mensaje te dice qué regla (por ejemplo `test_limit_of_active_loans`) no se está cumpliendo: revisa `rules.py`.

3.4 `services.py`: transacciones en Firestore

Aquí aparece un problema real: dos estudiantes pulsan **Prestar** a la vez sobre la **última copia** de un libro. Si cada servidor lee "queda 1 copia" y luego escribe, los dos se la llevan y el stock queda en -1.

La solución son las **transacciones**: Firestore garantiza que todo lo que ocurre dentro de la transacción sea atómico (todo o nada) y, si otro proceso modificó los datos mientras tanto, **repite** la transacción. Nuestras operaciones siguen siempre este patrón:

1. **Leer** (el libro, los préstamos activos del estudiante).
2. **Validar** con las funciones puras de `rules.py`.
3. **Escribir** (descontar la copia y crear el préstamo).

Detalles que debes notar al leer el código:

- `@firestore.transactional` convierte la función interna `run` en una transacción. Todas las **lecturas deben ir antes** que las escrituras.
- `firestore.Increment(-1)` resta una copia de forma atómica en el servidor, sin leer el valor antes.
- `_close()` sirve para devolver **y** cancelar: cambia el estado, guarda la fecha y repone la copia. `return_loan` y `cancel_loan` solo le indican cuál de las dos.
- `list_loans()` agrega el campo calculado `overdue` a cada préstamo y los ordena del más reciente al más antiguo.

features/loans/services.py — ARCHIVO NUEVO

```
"""Préstamos: prestar, devolver y cancelar (colección `loans` en Firestore).

Cada operación corre dentro de una transacción de Firestore: así dos personas
```

```
no pueden llevarse la última copia al mismo tiempo y el stock nunca queda mal.
Las reglas de negocio viven en `rules.py`.
"""
from datetime import timedelta

from django.conf import settings
from django.utils import timezone
from firebase_admin import firestore
from google.cloud.firestore_v1.base_query import FieldFilter

from core.firebase import get_db
from features.books.services import books_collection

from . import rules
from .rules import ACTIVE, CANCELLED, RETURNED, LoanError


def _loans():
    return get_db().collection('loans')


def _decorate(doc):
    loan = {'id': doc.id, **doc.to_dict()}
    loan['overdue'] = rules.is_overdue(loan, timezone.now())
    return loan


def list_loans(user_id=None, status=''):
    """Préstamos más recientes primero; opcionalmente de un usuario y/o estado."""
    query = _loans()
    if user_id:
        query = query.where(filter=FieldFilter('user_id', '==', user_id))
    loans = [_decorate(d) for d in query.stream()]
    if status == 'overdue':
        loans = [loan for loan in loans if loan['overdue']]
    elif status:
        loans = [loan for loan in loans if loan['status'] == status]
    return sorted(loans, key=lambda loan: loan['loaned_at'], reverse=True)


def borrow(book_id, user):
    """Presta un libro a `user` (dict de sesión). Valida stock y límites."""
    db = get_db()
    book_ref = books_collection().document(book_id)
    loan_ref = _loans().document()
    user_active = _loans().where(filter=FieldFilter('user_id', '==', user['uid'])).where(
        filter=FieldFilter('status', '==', ACTIVE)
    )

    @firestore.transactional
    def run(transaction):
        snapshot = book_ref.get(transaction=transaction)
        book = snapshot.to_dict() if snapshot.exists else None
        active = [d.to_dict() for d in user_active.stream(transaction=transaction)]
        rules.check_can_borrow(book, book_id, active)

        now = timezone.now()
        transaction.update(book_ref, {'copies_available': firestore.Increment(-1)})
        transaction.set(loan_ref, {
            'book_id': book_id, 'book_title': book['title'], 'book_category': book['category'],
            'user_id': user['uid'], 'user_name': user['name'], 'user_email': user['email'],
            'status': ACTIVE, 'loaned_at': now,
            'due_at': now + timedelta(days=settings.LOAN_DAYS),
            'returned_at': None, 'cancelled_at': None,
        })
```

```
    run(db.transaction())
    return loan_ref.id


def _close(loan_id, user, new_status, date_field):
    """Cierra un préstamo activo (devolver o cancelar) y repone la copia."""
    db = get_db()
    loan_ref = _loans().document(loan_id)

    @firestore.transactional
    def run(transaction):
        snapshot = loan_ref.get(transaction=transaction)
        loan = snapshot.to_dict() if snapshot.exists else None
        rules.check_can_close(loan, user)

        book_ref = books_collection().document(loan['book_id'])
        book_exists = book_ref.get(transaction=transaction).exists
        transaction.update(loan_ref, {'status': new_status, date_field: timezone.now()})
        if book_exists:
            transaction.update(book_ref, {'copies_available': firestore.Increment(1)})

    run(db.transaction())


def return_loan(loan_id, user):
    _close(loan_id, user, RETURNED, 'returned_at')


def cancel_loan(loan_id, user):
    _close(loan_id, user, CANCELLED, 'cancelled_at')
```

3.5 Vistas y rutas

Las vistas siguen siendo delgadas: reciben la petición, llaman al servicio y convierten el resultado en un mensaje (`messages.success` / `messages.error`) y una redirección.

Observaciones:

- Prestar y ver *Mis préstamos* son solo para estudiantes (`@student_required`); *Todos los préstamos* es solo para bibliotecarios.
- Devolver y cancelar usan `@login_required` porque pueden hacerlo **ambos** roles; la regla `check_can_close` decide si ese usuario puede con **ese** préstamo.
- Las operaciones que cambian datos solo aceptan `POST` (`@require_POST`).
- `_close` lee un campo oculto `next` para volver a la misma página (con el mismo número de página y filtros), y solo lo acepta si empieza por `/`.
- Las listas pasan por `paginate()`, igual que el catálogo.

features/loans/views.py — ARCHIVO NUEVO

```
from django.contrib import messages
from django.shortcuts import redirect, render
from django.views.decorators.http import require_POST

from core.decorators import librarian_required, login_required, student_required
from core.pagination import paginate

from . import services
```

```python
from .rules import LoanError


@student_required
@require_POST
def borrow(request, book_id):
    try:
        services.borrow(book_id, request.session['user'])
        messages.success(request, 'Préstamo registrado. ¡Disfruta la lectura!')
    except LoanError as exc:
        messages.error(request, str(exc))
    return redirect('loans:mine')


@login_required
@require_POST
def return_loan(request, loan_id):
    return _close(request, loan_id, services.return_loan, 'Libro devuelto.')


@login_required
@require_POST
def cancel_loan(request, loan_id):
    return _close(request, loan_id, services.cancel_loan, 'Préstamo cancelado.')


def _close(request, loan_id, action, success_message):
    try:
        action(loan_id, request.session['user'])
        messages.success(request, success_message)
    except LoanError as exc:
        messages.error(request, str(exc))
    next_url = request.POST.get('next', '')
    return redirect(next_url if next_url.startswith('/') else 'core:home')


@student_required
def my_loans(request):
    loans = services.list_loans(user_id=request.session['user']['uid'])
    return render(request, 'loans/mine.html', paginate(request, loans))


@librarian_required
def all_loans(request):
    status = request.GET.get('status', '')
    loans = services.list_loans(status=status)
    return render(request, 'loans/all.html', {**paginate(request, loans), 'status': status})
```

features/loans/urls.py — ARCHIVO NUEVO

```python
from django.urls import path

from . import views

app_name = 'loans'

urlpatterns = [
    path('', views.all_loans, name='all'),
    path('mine/', views.my_loans, name='mine'),
    path('borrow/<str:book_id>/', views.borrow, name='borrow'),
    path('<str:loan_id>/return/', views.return_loan, name='return'),
```

```
    path('<str:loan_id>/cancel/', views.cancel_loan, name='cancel'),
]
```

`<str:loan_id>` y `<str:book_id>` capturan el identificador desde la dirección y lo entregan a la vista. Las rutas fijas (`mine/`) y las que capturan datos conviven sin problema porque Django prueba las rutas de arriba hacia abajo y usa la primera que coincida.

3.6 Plantillas de `loans`

`_table.html` es un fragmento que dibuja la tabla de préstamos. Lo usan dos páginas, y se le pasa `show_user` para decidir si se muestra la columna del estudiante (el bibliotecario la ve; el estudiante no). Mira cómo cada estado tiene su insignia de color con un icono o texto (no dependemos solo del color).

features/loans/templates/loans/_table.html — ARCHIVO NUEVO

```
<figure>
<table class="table-cards">
  <thead>
    <tr>
      {% if show_user %}<th>Estudiante</th>{% endif %}
      <th>Libro</th><th>Prestado</th><th>Vence</th><th>Estado</th><th></th>
    </tr>
  </thead>
  <tbody>
  {% for loan in page_obj %}
    <tr>
      {% if show_user %}<td data-label="Estudiante"><div>{{ loan.user_name }}<br><small>{{ loan.user_email }}
</small></div></td>{% endif %}
      <td data-label="Libro">{{ loan.book_title }}</td>
      <td data-label="Prestado">{{ loan.loaned_at|date:"d M Y" }}</td>
      <td data-label="Vence">{{ loan.due_at|date:"d M Y" }}</td>
      <td data-label="Estado">
        {% if loan.overdue %}<span class="badge badge-overdue">⚠ Vencido</span>
        {% elif loan.status == 'active' %}<span class="badge badge-active">En curso</span>
        {% elif loan.status == 'returned' %}<span class="badge badge-returned">✓ Devuelto</span>
        {% else %}<span class="badge badge-cancelled">Cancelado</span>{% endif %}
      </td>
      <td class="cell-actions">
        {% if loan.status == 'active' %}
        <div class="actions">
          <form method="post" action="{% url 'loans:return' loan.id %}">{% csrf_token %}
            <input type="hidden" name="next" value="{{ request.get_full_path }}">
            <button type="submit">Devolver</button>
          </form>
          <form method="post" action="{% url 'loans:cancel' loan.id %}" onsubmit="return confirm('¿Cancelar
este préstamo?')">{% csrf_token %}
            <input type="hidden" name="next" value="{{ request.get_full_path }}">
            <button type="submit" class="outline secondary">Cancelar</button>
          </form>
        </div>
        {% endif %}
      </td>
    </tr>
  {% empty %}
    <tr><td colspan="6">No hay préstamos.</td></tr>
  {% endfor %}
  </tbody>
</table>
</figure>
```

features/loans/templates/loans/mine.html — ARCHIVO NUEVO

```
{% extends 'core/base.html' %}
{% block title %}Mis préstamos{% endblock %}
{% block content %}
<hgroup><h1>Mis préstamos</h1><p>Puedes tener hasta 3 préstamos activos a la vez.</p></hgroup>
{% include 'loans/_table.html' with show_user=False %}
{% include 'core/_pagination.html' %}
{% endblock %}
```

features/loans/templates/loans/all.html — ARCHIVO NUEVO

```
{% extends 'core/base.html' %}
{% block title %}Préstamos{% endblock %}
{% block content %}
<h1>Todos los préstamos</h1>
<form method="get" class="filter-form">
  <select name="status" onchange="this.form.submit()">
    <option value="">Todos los estados</option>
    <option value="active" {% if status == 'active' %}selected{% endif %}>En curso</option>
    <option value="overdue" {% if status == 'overdue' %}selected{% endif %}>Vencidos</option>
    <option value="returned" {% if status == 'returned' %}selected{% endif %}>Devueltos</option>
    <option value="cancelled" {% if status == 'cancelled' %}selected{% endif %}>Cancelados</option>
  </select>
</form>
{% include 'loans/_table.html' with show_user=True %}
{% include 'core/_pagination.html' %}
{% endblock %}
```

El selector de estado de `all.html` envía el formulario solo (`onchange="this.form.submit()"`) y la vista lo recibe como `?status=...`.

Estilos de la clase 3

Agrega al final de `site.css` las insignias de estado:

core/static/core/css/site.css — AGREGA AL FINAL

```
/* --- Insignias de estado ------------------------------------------------ */
.badge { display: inline-block; padding: .1rem .6rem; border-radius: 1rem; font-size: .8rem; font-weight: 600;
white-space: nowrap; }
.badge-active { background: #cde2fb; color: #0d366b; }
.badge-overdue { background: #f6d3d3; color: #7a1f1f; }
.badge-returned { background: #d3efd3; color: #064d06; }
.badge-cancelled { background: #e1e0d9; color: #52514e; }
```

3.7 Conectar `loans` con el proyecto

Cuatro cambios.

1. Registrar la app en `config/settings.py` (justo después de `books`):

MODIFICA `config/settings.py`

BUSCA este texto (está tal cual en el archivo):

```
    'features.books',
```

y REEMPLÁZALO por:

```
    'features.books',
    'features.loans',
```

2. Agregar la ruta en `config/urls.py`:

MODIFICA `config/urls.py`

BUSCA este texto (está tal cual en el archivo):

```
    path('books/', include('features.books.urls')),
```

y REEMPLÁZALO por:

```
    path('books/', include('features.books.urls')),
    path('loans/', include('features.loans.urls')),
```

3. Enlaces del menú en `core/templates/core/base.html`. El bibliotecario verá *Préstamos* y el estudiante *Mis préstamos*:

MODIFICA `core/templates/core/base.html`

BUSCA este texto (está tal cual en el archivo):

```
        <li><a href="{% url 'books:list' %}">Libros</a></li>
```

y REEMPLÁZALO por:

```
        <li><a href="{% url 'books:list' %}">Libros</a></li>
        {% if current_user.role == 'librarian' %}
          <li><a href="{% url 'loans:all' %}">Préstamos</a></li>
        {% else %}
          <li><a href="{% url 'loans:mine' %}">Mis préstamos</a></li>
        {% endif %}
```

4. El botón *Prestar* en `features/books/templates/books/list.html`. Reemplaza el comentario que dejamos en la clase 2. El botón se deshabilita cuando no quedan copias:

MODIFICA `features/books/templates/books/list.html`

BUSCA este texto (está tal cual en el archivo):

```
        {# Clase 3: aquí irá el botón "Prestar" #}
```

y REEMPLÁZALO por:

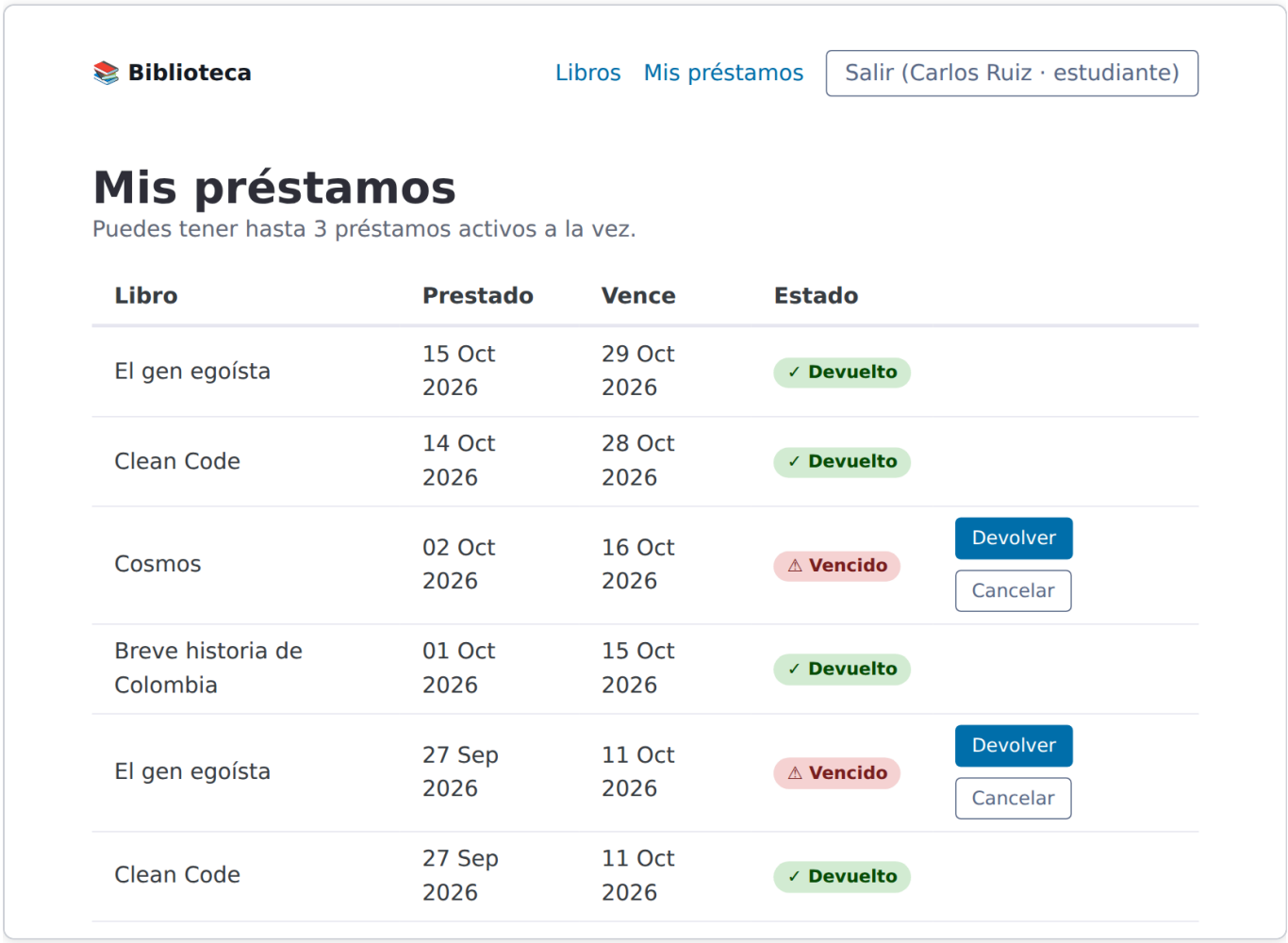
```
        <form method="post" action="{% url 'loans:borrow' book.id %}">{% csrf_token %}
          <button type="submit" {% if book.copies_available < 1 %}disabled{% endif %}>Prestar</button>
        </form>
```

3.8 Probar los préstamos

```
TERMINAL
python manage.py runserver
```

Como estudiante (elige uno de los que tienen préstamos activos):

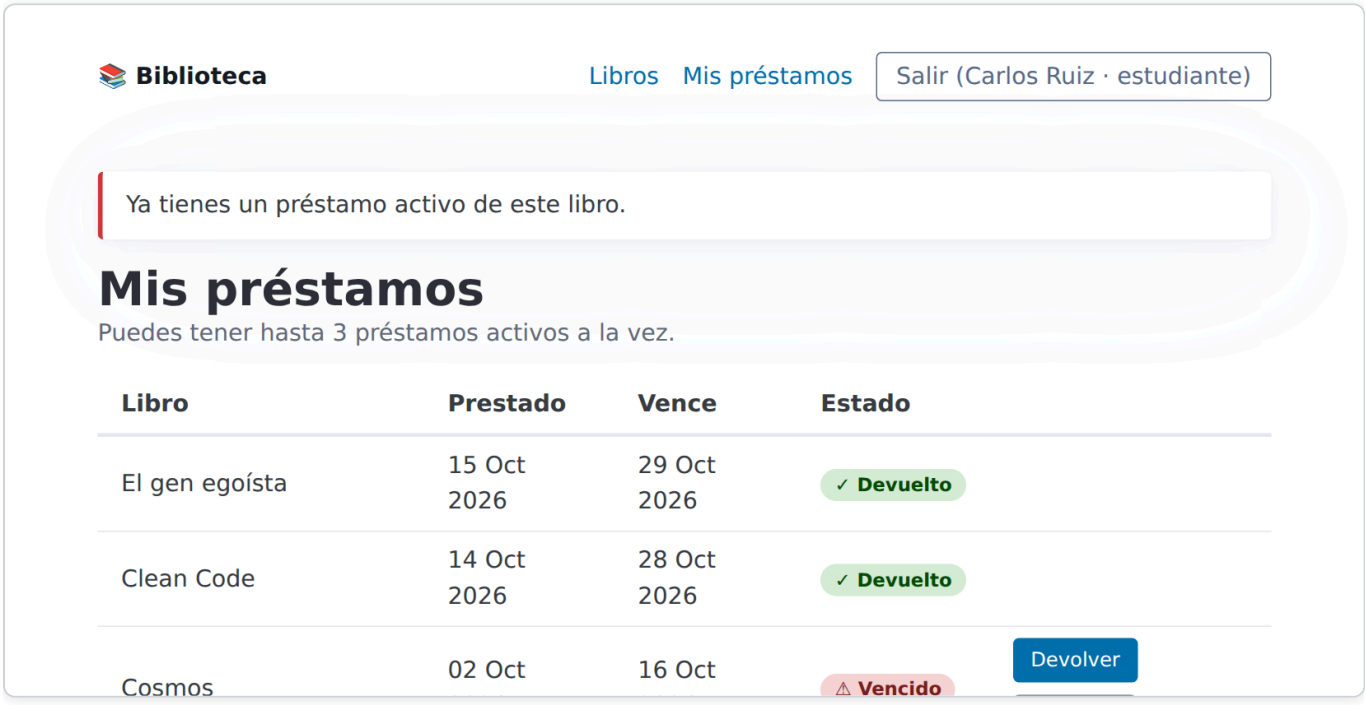
1. Abre **Mis préstamos**: verás tu historial paginado, con insignias y botones *Devolver* y *Cancelar* en los activos.
2. En el **Catálogo**, pulsa **Prestar** sobre un libro que ya tengas prestado: la regla lo rechaza con un mensaje.
3. Pulsa **Devolver** en un préstamo activo y comprueba que el mensaje cambia y la insignia pasa a *Devuelto*.
4. Intenta tener más de 3 préstamos activos: la cuarta vez verás *"Ya tienes 3 préstamos activos."*



Mis préstamos: historial paginado con insignias de estado y acciones



El botón Prestar en el catálogo del estudiante



La regla de negocio rechaza prestar un libro que ya tienes

Como bibliotecario:

1. Abre **Préstamos**: ves los de todos los estudiantes, con su nombre y correo.
2. Usa el selector de estado (*En curso*, *Vencidos*, *Devueltos*, *Cancelados*).
3. Devuelve o cancela un préstamo de un estudiante.



Vista del bibliotecario: todos los préstamos

> **✔ Verificación en la consola de Firebase**
>
> Abre la colección `loans` y mira un documento que acabas de devolver: `status` cambió a `returned` y `returned_at` tiene fecha. Abre el libro correspondiente en `books`: `copies_available` subió en 1.

3.9 Tests de las vistas

Ahora que `services.py` existe, completamos `features/loans/tests.py`. Primero importa `services` (cambia la línea de importación de arriba):

MODIFICA `features/loans/tests.py`

BUSCA este texto (está tal cual en el archivo):

```
from . import rules
```

y REEMPLÁZALO por:

```
from . import rules, services
```

Y luego agrega las pruebas de las vistas al **final** del archivo. Cubren los permisos (un bibliotecario **no puede** prestar), los mensajes de error, que `next` no acepte direcciones externas y que la paginación conserve el filtro de estado.

features/loans/tests.py — AGREGA AL FINAL

```
class LoanViewTests(SessionTestCase):
    def test_student_borrows(self):
        self.login_as(STUDENT)
        with mock.patch.object(services, 'borrow') as borrow, \
                mock.patch.object(services, 'list_loans', return_value=[]):
            response = self.client.post('/loans/borrow/b1/', follow=True)
        borrow.assert_called_once_with('b1', STUDENT)
        self.assertContains(response, 'Préstamo registrado')

    def test_business_error_is_shown_to_the_user(self):
        self.login_as(STUDENT)
        with mock.patch.object(services, 'borrow', side_effect=LoanError('No hay copias disponibles de este
libro.')), \
                mock.patch.object(services, 'list_loans', return_value=[]):
            response = self.client.post('/loans/borrow/b1/', follow=True)
        self.assertContains(response, 'No hay copias disponibles')

    def test_librarian_cannot_borrow(self):
        self.login_as(LIBRARIAN)
        with mock.patch.object(services, 'borrow') as borrow:
            self.client.post('/loans/borrow/b1/')
        borrow.assert_not_called()

    def test_borrow_requires_post(self):
        self.login_as(STUDENT)
        self.assertEqual(self.client.get('/loans/borrow/b1/').status_code, 405)

    def test_return_and_cancel_use_the_session_user(self):
        self.login_as(STUDENT)
        with mock.patch.object(services, 'return_loan') as ret, mock.patch.object(services, 'cancel_loan') as
can:
            self.client.post('/loans/l1/return/')
            self.client.post('/loans/l1/cancel/')
        ret.assert_called_once_with('l1', STUDENT)
        can.assert_called_once_with('l1', STUDENT)

    def test_next_redirect_ignores_external_urls(self):
        self.login_as(STUDENT)
        with mock.patch.object(services, 'return_loan'):
            ok = self.client.post('/loans/l1/return/', {'next': '/loans/mine/?page=2'})
            bad = self.client.post('/loans/l1/return/', {'next': 'https://evil.example/'})
        self.assertEqual(ok['Location'], '/loans/mine/?page=2')
        self.assertEqual(bad['Location'], '/')

    def test_my_loans_only_asks_for_own_loans_and_paginates(self):
        self.login_as(STUDENT)
        many = [loan(id=f'l{i}', book_title=f'Libro {i:02}') for i in range(25)]
        with mock.patch.object(services, 'list_loans', return_value=many) as list_loans:
            response = self.client.get('/loans/mine/?page=2')
        list_loans.assert_called_once_with(user_id=STUDENT['uid'])
        self.assertContains(response, 'Libro 10')
        self.assertNotContains(response, 'Libro 09')
        self.assertContains(response, 'Página 2 de 3')

    def test_all_loans_keeps_status_filter_in_pagination(self):
        self.login_as(LIBRARIAN)
        many = [loan(id=f'l{i}') for i in range(25)]
        with mock.patch.object(services, 'list_loans', return_value=many) as list_loans:
```

```
            response = self.client.get('/loans/?status=active')
        list_loans.assert_called_once_with(status='active')
        self.assertContains(response, '?status=active&page=2')
```

Y la prueba que dejamos pendiente en la clase 2: que el estudiante vea el botón *Prestar* y el bibliotecario no. Agrégala en `features/books/tests.py`, justo antes de `test_student_cannot_create_or_delete_books`:

MODIFICA `features/books/tests.py`

BUSCA este texto (está tal cual en el archivo):

```
    def test_student_cannot_create_or_delete_books(self):
```

y REEMPLÁZALO por:

```
    def test_students_see_borrow_button_and_librarians_see_edit(self):
        with mock.patch.object(services, 'list_books', return_value=make_books(1)), \
                mock.patch.object(services, 'list_categories', return_value=[]):
            self.login_as(STUDENT)
            self.assertContains(self.client.get('/books/'), 'Prestar')
            self.login_as(LIBRARIAN)
            response = self.client.get('/books/')
        self.assertContains(response, 'Editar')
        self.assertNotContains(response, 'Prestar')

    def test_student_cannot_create_or_delete_books(self):
```

TERMINAL

```
python manage.py test
```

SALIDA ESPERADA

```
Found 52 test(s).
System check identified no issues (0 silenced).
Creating test database for alias 'default'...
....................................................
----------------------------------------------------------------------
Ran 52 tests in 0.220s

OK
Destroying test database for alias 'default'...
```

✔ Verificación de la clase 3

- `python manage.py test` termina con **OK** (52 pruebas).
- El estudiante presta, devuelve y cancela; las reglas lo frenan cuando corresponde.
- El bibliotecario ve todos los préstamos y puede filtrarlos.
- Después de cada operación, `copies_available` cuadra con los préstamos activos.

3.10 Errores comunes de la clase 3

Mensaje o síntoma	Causa probable	Solución
`NoReverseMatch: 'loans' is not a registered namespace`	Falta registrar la ruta o el `app_name = 'loans'`	Revisa `config/urls.py` y `features/loans/urls.py`
`ImportError: cannot import name 'books_collection'`	`features/books/services.py` no tiene esa función	Compara con la clase 2
`400 The query requires an index`	Una consulta de Firestore combina filtros y orden	Nuestras consultas ordenan en Python; revisa que no hayas agregado `order_by`
La transacción falla con `read after write`	Escribiste en Firestore antes de terminar de leer	Todas las lecturas (`.get` , `.stream`) van **antes** de `transaction.update/set`
Al pulsar *Prestar* no pasa nada	El botón está dentro de un `<form>` sin `{% csrf_token %}`	Django rechaza el `POST` sin token; agrega la etiqueta
`405 Method Not Allowed`	Entraste por la barra de direcciones a una ruta que solo acepta `POST`	Es lo esperado; usa el botón
Un bibliotecario ve *"No tienes permiso"* en `/loans/mine/`	Correcto: esa página es solo de estudiantes	Usa *Préstamos*
`copies_available` negativo	Se modificó el stock fuera de las transacciones	Vuelve a sembrar con `python manage.py seed`

CLASE 4

Dashboard, diseño responsive y cierre

Duración estimada: 3 horas · **Al terminar:** el bibliotecario tiene un dashboard con indicadores y cuatro gráficas, todo el sitio se ve bien en celular, tablet y computador, y tu proyecto tiene **71 pruebas** en verde.

> **Antes de empezar**
>
> Activa el entorno virtual y comprueba que `python manage.py test` termina en **OK** (52 pruebas) y que hay datos sembrados (`python manage.py seed`).

4.1 Qué vamos a hacer hoy

1. Calcular **estadísticas** con una función pura y probarla.
2. Dibujar **gráficas** con Plotly, todo desde Python.
3. Crear la feature `dashboard` y conectarla al proyecto.
4. Hacer el sitio **responsive** con CSS.
5. Completar las pruebas y revisar el proyecto terminado.

4.2 `stats.py`: de datos a números

El dashboard responde preguntas como: ¿cuántos préstamos hay por mes?, ¿qué libros se piden más?, ¿cuántos están vencidos? Para eso separamos dos cosas:

- `compute_stats(loans, books)` es una **función pura**: recibe listas y devuelve un diccionario con los números. No toca Firestore, así que se prueba fácil.
- `dashboard_stats()` solo **lee** los datos de los servicios y se los pasa a la anterior.

Usa `collections.Counter`, una herramienta de Python que cuenta repeticiones: `Counter(titulos).most_common(8)` devuelve los 8 títulos más frecuentes. `last_months()` calcula los últimos 6 meses (cruzando el cambio de año) para que los meses sin préstamos aparezcan con 0.

TERMINAL

```
mkdir -p features/dashboard/templates/dashboard
touch features/dashboard/__init__.py
```

features/dashboard/apps.py — ARCHIVO NUEVO

```
from django.apps import AppConfig
```

```python
class DashboardConfig(AppConfig):
    name = 'features.dashboard'
```

features/dashboard/stats.py — ARCHIVO NUEVO

```python
"""Estadísticas del dashboard.

`compute_stats` es una función pura (recibe listas, devuelve números) y se prueba sin
Firestore. `dashboard_stats` solo lee los datos y se los pasa.
"""
from collections import Counter

from django.utils import timezone

from features.books import services as books_service
from features.loans import services as loans_service

MONTHS_ES = ['ene', 'feb', 'mar', 'abr', 'may', 'jun', 'jul', 'ago', 'sep', 'oct', 'nov', 'dic']


def last_months(count, now):
    """Últimos `count` meses como lista de (año, mes), el más antiguo primero."""
    now = timezone.localtime(now)
    year, month = now.year, now.month
    months = []
    for _ in range(count):
        months.append((year, month))
        month -= 1
        if month == 0:
            year, month = year - 1, 12
    return months[::-1]


def compute_stats(loans, books, now=None, months=6):
    now = now or timezone.now()

    status_counts = Counter()
    for loan in loans:
        key = 'overdue' if loan['overdue'] else loan['status']
        status_counts[key] += 1

    by_month = Counter()
    for loan in loans:
        local = timezone.localtime(loan['loaned_at'])
        by_month[(local.year, local.month)] += 1
    month_series = [(f'{MONTHS_ES[m - 1]} {y}', by_month[(y, m)]) for y, m in last_months(months, now)]

    return {
        'kpis': {
            'books': len(books),
            'copies_available': sum(b['copies_available'] for b in books),
            'loans': len(loans),
            'active': status_counts['active'] + status_counts['overdue'],
            'overdue': status_counts['overdue'],
        },
        'status_counts': status_counts,
        'month_series': month_series,
        'top_books': Counter(loan['book_title'] for loan in loans).most_common(8),
        'by_category': Counter(loan['book_category'] for loan in loans).most_common(),
    }
```

```python
def dashboard_stats(months=6):
    return compute_stats(loans_service.list_loans(), books_service.list_books(), months=months)
```

4.3 `charts.py`: gráficas con Plotly

Plotly dibuja gráficas interactivas (con tooltip al pasar el cursor) y las entrega como HTML. Nosotros **no escribimos JavaScript**: construimos la figura en Python y la incrustamos en la plantilla.

Decisiones de diseño que vale la pena entender:

- **Un solo color (azul)** para magnitudes como "préstamos por mes". Los colores distintos se reservan para los **estados** (rojo = vencido, verde = devuelto, gris = cancelado), así el color siempre significa algo.
- **Barras horizontales** cuando las etiquetas son largas (títulos de libros). Los títulos largos se parten en varias líneas con `textwrap` para dejar espacio a las barras.
- `_headroom()` agrega un 20 % de espacio sobre la barra más larga para que la etiqueta con su número no se corte en el borde.
- La gráfica de estados llega hasta 100 como mínimo (`max(100, ...)`).
- `_html()` envuelve cada gráfica en un `<div>` con **alto fijo**. Sin él, Plotly mide 0 de alto y las gráficas se montan unas sobre otras. Además agrega un pequeño script (`post_script`) que reajusta el tamaño al terminar de cargar la página, para que ninguna gráfica se desborde de su tarjeta.
- La librería `plotly.js` (el JavaScript que dibuja) se carga **una sola vez** por página: solo la primera gráfica lleva `first=True`.

features/dashboard/charts.py — ARCHIVO NUEVO

```python
"""Gráficas del dashboard con Plotly, generadas 100 % en Python."""
import textwrap

import plotly.graph_objects as go
from django.utils.safestring import mark_safe

# Paleta: un solo tono (azul) para magnitudes; colores de estado solo para estados.
BLUE = '#2a78d6'
STATUS_COLORS = {
    'active': ('En curso', '#2a78d6'),
    'overdue': ('Vencidos', '#d03b3b'),
    'returned': ('Devueltos', '#0ca30c'),
    'cancelled': ('Cancelados', '#898781'),
}
INK, MUTED, GRID = '#0b0b0b', '#52514e', '#e1e0d9'


def _headroom(values):
    """Rango [0, máximo + 20 %] para que la etiqueta de la barra más larga no se corte."""
    return [0, max(max(values, default=0), 1) * 1.2]


def _layout(fig, height=320):
    fig.update_layout(
        height=height,
        margin=dict(l=10, r=30, t=10, b=10),
        paper_bgcolor='rgba(0,0,0,0)',
        plot_bgcolor='rgba(0,0,0,0)',
        font=dict(family='system-ui, -apple-system, "Segoe UI", sans-serif', color=MUTED, size=13),
        showlegend=False,
        bargap=0.45,
```

```
        hoverlabel=dict(bgcolor='#fcfcfb', font_color=INK),
    )
    fig.update_xaxes(showgrid=False, zeroline=False, linecolor=GRID)
    fig.update_yaxes(gridcolor=GRID, zeroline=False, showline=False)
    return fig


def _html(fig, first):
    """Convierte la figura a HTML. Plotly.js se carga por CDN una sola vez."""
    chart = fig.to_html(
        full_html=False,
        include_plotlyjs='cdn' if first else False,
        config={'displayModeBar': False, 'responsive': True},
        # Plotly mide el contenedor mientras la página aún se acomoda y puede dibujar la
        # figura más ancha que su tarjeta. Al terminar de cargar la ajustamos al ancho real.
        post_script='window.addEventListener("load", function () { Plotly.Plots.resize("{plot_id}"); });',
    )
    # Plotly dibuja la figura en un div con height:100%. Si el contenedor no tiene
    # un alto fijo, mide 0 y las gráficas se montan unas sobre otras.
    return mark_safe(f'<div style="height:{fig.layout.height}px">{chart}</div>')


def loans_per_month(series, first=False):
    fig = go.Figure(go.Bar(
        x=[label for label, _ in series], y=[n for _, n in series],
        marker_color=BLUE, text=[n for _, n in series], textposition='outside',
        hovertemplate='%{x}: %{y} préstamos<extra></extra>', cliponaxis=False,
    ))
    fig.update_yaxes(range=_headroom(n for _, n in series))
    return _html(_layout(fig), first)


def loans_by_status(status_counts, first=False):
    order = ['active', 'overdue', 'returned', 'cancelled']
    fig = go.Figure(go.Bar(
        y=[STATUS_COLORS[k][0] for k in order],
        x=[status_counts[k] for k in order],
        orientation='h',
        marker_color=[STATUS_COLORS[k][1] for k in order],
        text=[status_counts[k] for k in order], textposition='outside',
        hovertemplate='%{y}: %{x}<extra></extra>', cliponaxis=False,
    ))
    fig.update_yaxes(autorange='reversed', gridcolor='rgba(0,0,0,0)')
    fig.update_xaxes(showgrid=True, gridcolor=GRID, range=[0, max(100, _headroom(status_counts.values())[1])])
    return _html(_layout(fig, height=260), first)


def top_books(pairs, first=False):
    pairs = pairs[::-1]  # el más prestado arriba
    fig = go.Figure(go.Bar(
        # títulos largos en varias líneas para dejar espacio a las barras en pantallas angostas
        y=['<br>'.join(textwrap.wrap(t, 22)) for t, _ in pairs], x=[n for _, n in pairs],
        orientation='h', marker_color=BLUE,
        text=[n for _, n in pairs], textposition='outside', cliponaxis=False,
        customdata=[t for t, _ in pairs],
        hovertemplate='%{customdata}: %{x} préstamos<extra></extra>',
    ))
    fig.update_yaxes(gridcolor='rgba(0,0,0,0)', automargin=True)
    fig.update_xaxes(showgrid=True, gridcolor=GRID, range=_headroom(n for _, n in pairs))
    return _html(_layout(fig, height=420), first)


def loans_by_category(pairs, first=False):
    fig = go.Figure(go.Bar(
        x=[c for c, _ in pairs], y=[n for _, n in pairs],
        marker_color=BLUE, text=[n for _, n in pairs], textposition='outside',
```

```
        hovertemplate='%{x}: %{y} préstamos<extra></extra>', cliponaxis=False,
    ))
    fig.update_yaxes(range=_headroom(n for _, n in pairs))
    return _html(_layout(fig), first)
```

4.4 Vista, ruta y plantilla del dashboard

La vista pide las estadísticas, construye las cuatro gráficas y se las entrega a la plantilla. Solo los bibliotecarios pueden entrar.

features/dashboard/views.py — ARCHIVO NUEVO

```
from django.shortcuts import render

from core.decorators import librarian_required

from . import charts, stats


@librarian_required
def index(request):
    s = stats.dashboard_stats()
    return render(request, 'dashboard/index.html', {
        'kpis': s['kpis'],
        'chart_months': charts.loans_per_month(s['month_series'], first=True),
        'chart_status': charts.loans_by_status(s['status_counts']),
        'chart_top': charts.top_books(s['top_books']),
        'chart_category': charts.loans_by_category(s['by_category']),
    })
```

features/dashboard/urls.py — ARCHIVO NUEVO

```
from django.urls import path

from . import views

app_name = 'dashboard'

urlpatterns = [
    path('', views.index, name='index'),
]
```

La plantilla muestra cinco **indicadores** (KPI) y las cuatro gráficas. Las gráficas llegan ya como HTML, por eso aquí se escriben simplemente `{{ chart_months }}`. (Django marca ese HTML como seguro con `mark_safe`, dentro de `charts.py`.)

features/dashboard/templates/dashboard/index.html — ARCHIVO NUEVO

```
{% extends 'core/base.html' %}
{% block title %}Dashboard{% endblock %}
{% block content %}
<h1>Dashboard</h1>
<div class="kpis">
  <article><strong>{{ kpis.books }}</strong><small>Libros</small></article>
  <article><strong>{{ kpis.copies_available }}</strong><small>Copias disponibles</small></article>
  <article><strong>{{ kpis.loans }}</strong><small>Préstamos totales</small></article>
```

```
  <article><strong>{{ kpis.active }}</strong><small>Préstamos activos</small></article>
  <article><strong>{{ kpis.overdue }}</strong><small>⚠ Vencidos</small></article>
</div>
<br>
<div class="charts">
  <article><header>Préstamos por mes</header>{{ chart_months }}</article>
  <article><header>Préstamos por estado</header>{{ chart_status }}</article>
  <article><header>Libros más prestados</header>{{ chart_top }}</article>
  <article><header>Préstamos por categoría</header>{{ chart_category }}</article>
</div>
{% endblock %}
```

Estilos del dashboard (al final de `site.css`):

core/static/core/css/site.css — AGREGA AL FINAL

```
/* --- Dashboard ------------------------------------------------------------ */
.kpis { display: grid; grid-template-columns: repeat(auto-fit, minmax(120px, 1fr)); gap: 1rem; }
.kpis article { margin: 0; text-align: center; }
.kpis strong { display: block; font-size: 2rem; line-height: 1.2; }
.charts { display: grid; grid-template-columns: repeat(2, minmax(0, 1fr)); gap: 1rem; }
.charts article { margin: 0; min-width: 0; overflow: hidden; }
```

Conectar el dashboard

1. Registrar la app en `config/settings.py` (después de `loans`):

MODIFICA `config/settings.py`

BUSCA este texto (está tal cual en el archivo):

```
    'features.loans',
```

y REEMPLÁZALO por:

```
    'features.loans',
    'features.dashboard',
```

2. La ruta en `config/urls.py`:

MODIFICA `config/urls.py`

BUSCA este texto (está tal cual en el archivo):

```
    path('loans/', include('features.loans.urls')),
```

y REEMPLÁZALO por:

```
    path('loans/', include('features.loans.urls')),
    path('dashboard/', include('features.dashboard.urls')),
```

3. El enlace del menú en `core/templates/core/base.html`:

MODIFICA `core/templates/core/base.html`

BUSCA este texto (está tal cual en el archivo):

```
            <li><a href="{% url 'loans:all' %}">Préstamos</a></li>
```

y REEMPLÁZALO por:

```
            <li><a href="{% url 'loans:all' %}">Préstamos</a></li>
            <li><a href="{% url 'dashboard:index' %}">Dashboard</a></li>
```

4. La página de inicio final. Ahora sí: el bibliotecario aterriza en el dashboard y el estudiante en el catálogo. **Reemplaza todo** `core/views.py`:

core/views.py — REEMPLAZA TODO EL ARCHIVO

```
from django.shortcuts import redirect

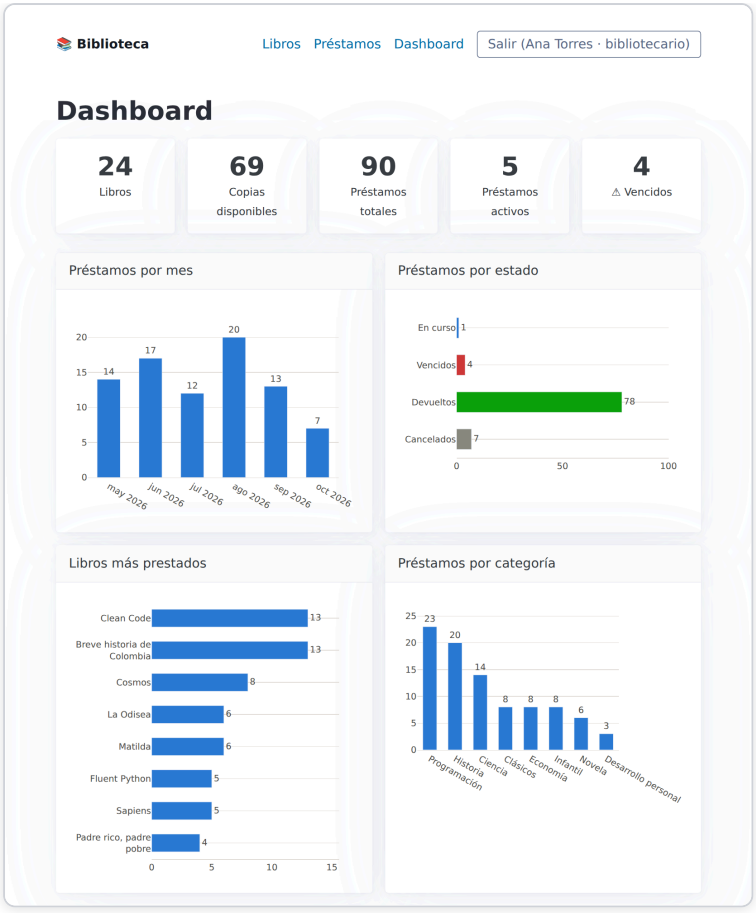

def home(request):
    """Cada rol aterriza en su página principal."""
    user = request.session.get('user')
    if user and user['role'] == 'librarian':
        return redirect('dashboard:index')
    return redirect('books:list')
```

4.5 Probar el dashboard

TERMINAL

```
python manage.py runserver
```

Ingresa con el bibliotecario: ahora `/` te lleva al dashboard. Pasa el cursor sobre las barras para ver el detalle de cada una. Si ingresas con un estudiante y escribes `/dashboard/`, te devuelve al catálogo con el mensaje *"No tienes permiso para ver esa página."*



Dashboard del bibliotecario

4.6 Diseño responsive

Hoy casi la mitad de las visitas web vienen de un celular. **Responsive** significa que la misma página se adapta al tamaño de la pantalla. Se logra con tres ideas que ya tienes en el proyecto o vas a agregar:

1. **La etiqueta `viewport`** en `base.html`: `<meta name="viewport" content="width=device-width, initial-scale=1">`. Sin ella, el celular dibuja la página como si midiera 980 px y la encoge.
2. **Anchos flexibles:** usamos `flex` y `grid` en lugar de anchos fijos, para que los elementos se acomoden solos.
3. **Media queries:** reglas CSS que se aplican **solo** cuando la pantalla cumple una condición, por ejemplo `@media (max-width: 768px) { ... }` ("si la pantalla mide 768 px o menos").

Nuestros tres tamaños de prueba:

Dispositivo	Ancho	Qué cambia
Celular	**360 px**	Menú en dos filas, tablas convertidas en tarjetas, indicadores en 2 columnas, gráficas en 1 columna, paginación solo con *Anterior / Siguiente*
Tablet	**768 px**	Igual que el celular, pero con los indicadores en una sola fila y las gráficas en 2 columnas
Computador	**1024 px** o más	Tablas completas, menú en una fila, gráficas en cuadrícula de 2 × 2

La técnica estrella: tablas que se vuelven tarjetas

Una tabla de 5 columnas no cabe en 360 px. En vez de hacer scroll horizontal, en pantallas pequeñas convertimos **cada fila en una tarjeta**:

- Ocultamos el encabezado (`<thead>`).
- Hacemos que cada `<tr>` y `<td>` sea un bloque.
- Cada celda muestra su **etiqueta** gracias a `td::before { content: attr(data-label); }` : CSS lee el atributo `data-label="Autor"` que ya pusimos en el HTML y lo dibuja antes del valor.

Por eso en las plantillas de las clases 2 y 3 escribimos `<td data-label="Autor">`. Ahora le damos los estilos. Agrega al **final** de `site.css` estas dos secciones:

core/static/core/css/site.css — AGREGA AL FINAL

```
/* --- Responsive: tablet (≤ 768px) ---------------------------------------- */
@media (max-width: 768px) {
  /* Menú en dos filas: marca arriba, enlaces y "Salir" debajo */
  nav { flex-direction: column; align-items: flex-start; row-gap: 0; }
  nav li { padding-top: .35rem; padding-bottom: .35rem; }
  nav ul:last-child { margin-left: calc(var(--pico-nav-element-spacing-horizontal) * -1); }

  /* Las tablas pasan a tarjetas: cada fila es una tarjeta y cada celda "etiqueta: valor" */
  .table-cards thead { position: absolute; width: 1px; height: 1px; overflow: hidden; clip-path: inset(50%); }
  .table-cards, .table-cards tbody, .table-cards tr, .table-cards td { display: block; width: 100%; }
  .table-cards tr {
    margin-bottom: 1rem; padding: .4rem .9rem;
    border: 1px solid var(--pico-muted-border-color); border-radius: var(--pico-border-radius);
  }
  .table-cards td {
    display: flex; justify-content: space-between; align-items: center; gap: 1rem;
    padding: .4rem 0; border: 0; text-align: right;
  }
  .table-cards td + td { border-top: 1px solid var(--pico-muted-border-color); }
  .table-cards td::before { content: attr(data-label); flex: none; font-weight: 600; text-align: left; color:
var(--pico-muted-color); }
  .table-cards td > div { min-width: 0; overflow-wrap: anywhere; }
  .table-cards .cell-actions:not(:has(.actions)) { display: none; }  /* sin botones, sin celda */
  .table-cards .cell-actions .actions { width: 100%; }
  .table-cards .cell-actions .actions > * { flex: 1; }
  .table-cards .cell-actions .actions button, .table-cards .cell-actions .actions a { width: 100%; padding:
.5rem; }

  .filter-form { max-width: none; }
}

/* --- Responsive: móvil (≤ 700px y ≤ 480px) ------------------------------- */
@media (max-width: 700px) {
  .charts { grid-template-columns: minmax(0, 1fr); }
}
@media (max-width: 480px) {
  .kpis { grid-template-columns: repeat(2, minmax(0, 1fr)); gap: .75rem; }
  .kpis article:last-child:nth-child(odd) { grid-column: 1 / -1; }
  .kpis strong { font-size: 1.6rem; }
  .nav-role { display: none; }
  .search-form > * { flex: 1 1 100%; }
  h1 { font-size: 1.7rem; }
  .auth-card, .form-card { margin-top: .5rem; }
}
```

Detalles de CSS que verás en el código

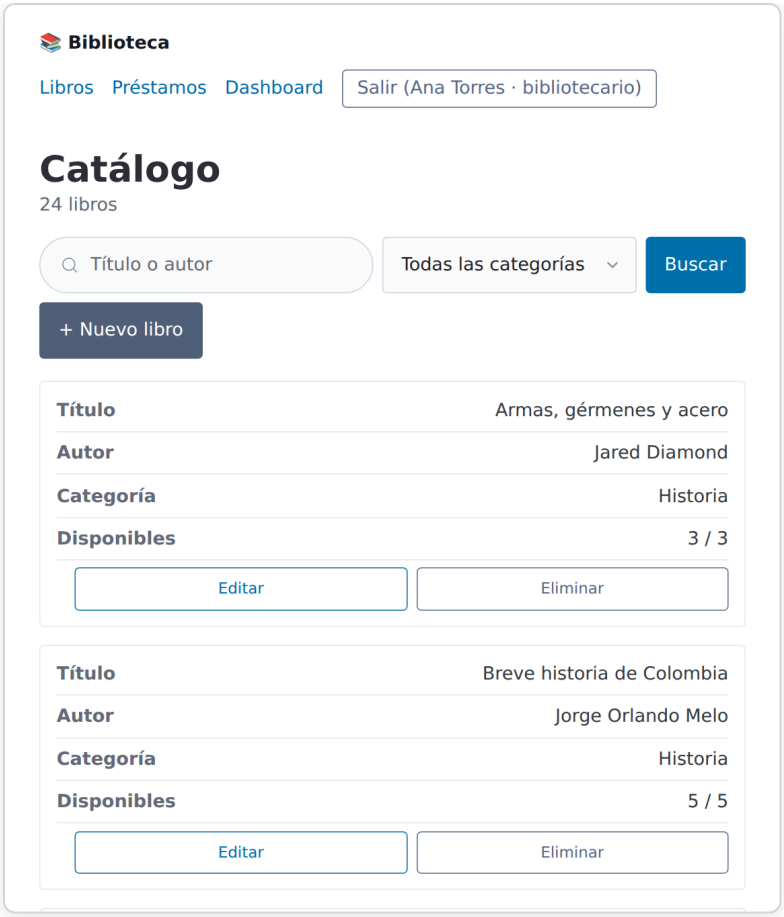
- `:is(input, select, button)` agrupa varios elementos en una sola regla.
- `:has(.actions)` selecciona una celda **que contiene** botones, así ocultamos las celdas vacías en las tarjetas.
- `var(--pico-...)` son variables de color y espaciado que define Pico CSS; usarlas mantiene el diseño coherente.
- `clip-path: inset(50%)` oculta el encabezado visualmente pero lo deja disponible para lectores de pantalla (accesibilidad).

Cómo probar tamaños en tu navegador

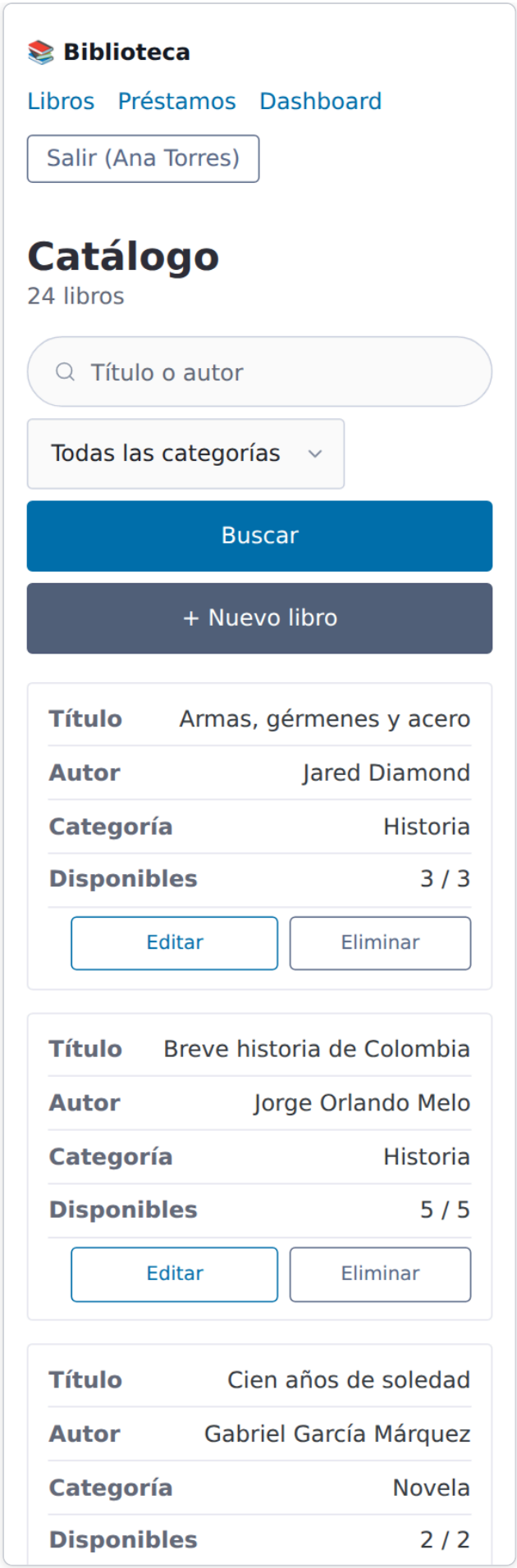
1. Abre una página del sitio y presiona **F12** (herramientas de desarrollador).
2. Activa el **modo dispositivo**: **Ctrl + Shift + M** (en Mac, **Cmd + Shift + M**).
3. Arriba aparece la barra de tamaño: escribe **360**, luego **768**, luego **1024** y recarga con **Ctrl + R**.
4. Revisa que **no aparezca scroll horizontal** en ningún tamaño y que nada se corte.



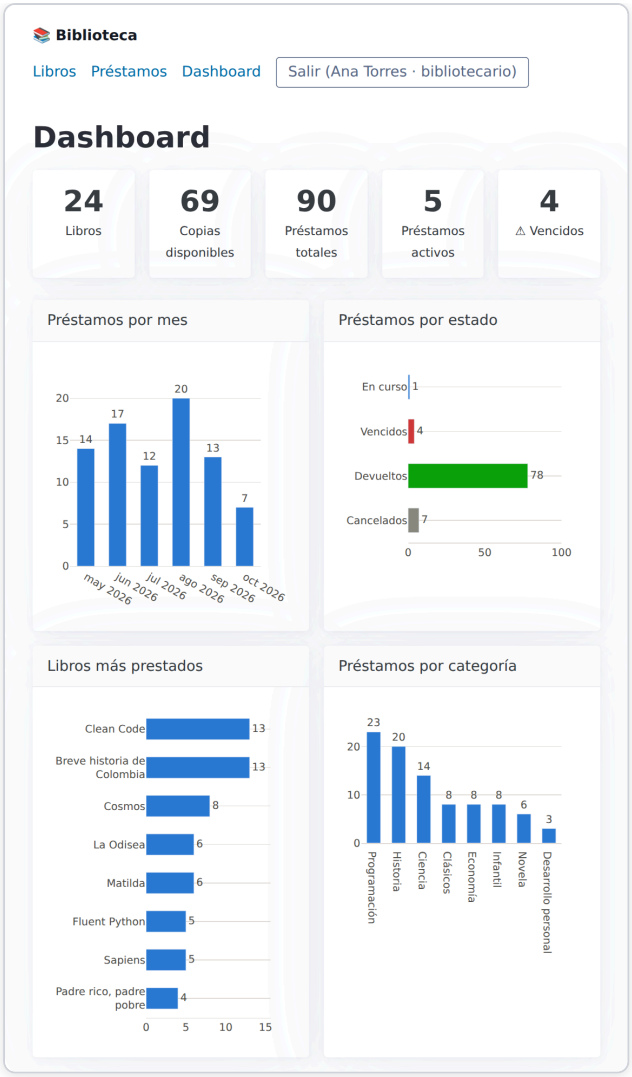
Catálogo a 1024 px: tabla completa



Catálogo a 768 px: tarjetas y menú en dos filas



Catálogo a 360 px (celular)



Dashboard a 768 px

📚 **Biblioteca**

Libros Préstamos Dashboard

Salir (Ana Torres)

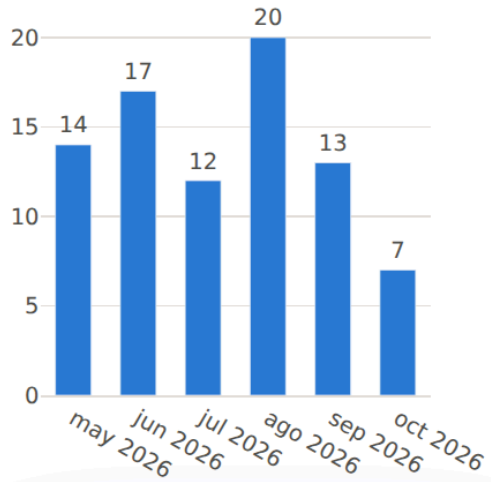
Dashboard

24	69
Libros	Copias disponibles
90	**5**
Préstamos totales	Préstamos activos

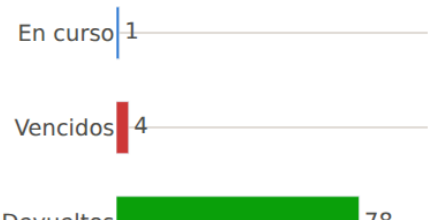
4

⚠ Vencidos

Préstamos por mes



Préstamos por estado





Dashboard a 360 px

📚 **Biblioteca**

Libros Mis préstamos

Salir (Carlos Ruiz)

Mis préstamos

Puedes tener hasta 3 préstamos activos a la vez.

Libro	El gen egoísta
Prestado	15 Oct 2026
Vence	29 Oct 2026
Estado	✓ **Devuelto**

Libro	Clean Code
Prestado	14 Oct 2026
Vence	28 Oct 2026
Estado	✓ **Devuelto**

Libro	Cosmos
Prestado	02 Oct 2026
Vence	16 Oct 2026
Estado	⚠ **Vencido**

Devolver Cancelar

Libro	Breve historia de Colombia
Prestado	01 Oct 2026
Vence	15 Oct 2026
Estado	✓ **Devuelto**

Libro	El gen egoísta
Prestado	27 Sep 2026

4.7 Tests de la clase 4

`features/dashboard/tests.py` comprueba tres cosas: que `compute_stats` cuenta bien (incluido el cruce de año), que las gráficas se generan con las dimensiones esperadas aunque no haya datos, y que solo el bibliotecario entra a la página.

features/dashboard/tests.py — ARCHIVO NUEVO

```
from datetime import datetime, timedelta, timezone as dt_timezone
from unittest import mock

from django.test import SimpleTestCase

from core.testing import LIBRARIAN, STUDENT, SessionTestCase

from . import charts, stats

NOW = datetime(2026, 10, 16, 12, 0, tzinfo=dt_timezone.utc)


def loan(title='Clean Code', category='Programación', status='returned', overdue=False, days_ago=3):
    return {'book_title': title, 'book_category': category, 'status': status, 'overdue': overdue,
            'loaned_at': NOW - timedelta(days=days_ago)}


BOOKS = [
    {'copies_available': 2, 'copies_total': 3},
    {'copies_available': 0, 'copies_total': 1},
]


class ComputeStatsTests(SimpleTestCase):
    def setUp(self):
        self.loans = [
            loan(status='active'),
            loan(status='active', overdue=True, days_ago=20),
            loan(status='returned', days_ago=40),
            loan(title='Cosmos', category='Ciencia', status='cancelled', days_ago=70),
            loan(title='Clean Code', status='returned', days_ago=100),
        ]
        self.s = stats.compute_stats(self.loans, BOOKS, now=NOW)

    def test_kpis(self):
        self.assertEqual(self.s['kpis'], {'books': 2, 'copies_available': 2, 'loans': 5, 'active': 2,
'overdue': 1})

    def test_overdue_is_counted_apart_from_active(self):
        self.assertEqual(self.s['status_counts']['active'], 1)
        self.assertEqual(self.s['status_counts']['overdue'], 1)

    def test_months_series_has_six_entries_ending_this_month(self):
        labels = [label for label, _ in self.s['month_series']]
        self.assertEqual(len(labels), 6)
        self.assertEqual(labels[-1], 'oct 2026')
        self.assertEqual(labels[0], 'may 2026')

    def test_months_without_loans_count_zero(self):
        by_label = dict(self.s['month_series'])
```

```
        self.assertEqual(sum(by_label.values()), 5)
        self.assertEqual(stats.compute_stats([], BOOKS, now=NOW)['month_series'][0][1], 0)

    def test_top_books_ordered_by_loans(self):
        self.assertEqual(self.s['top_books'][0], ('Clean Code', 4))

    def test_by_category(self):
        self.assertEqual(dict(self.s['by_category']), {'Programación': 4, 'Ciencia': 1})

    def test_last_months_crosses_the_year(self):
        jan = datetime(2026, 1, 15, tzinfo=dt_timezone.utc)
        self.assertEqual(stats.last_months(3, jan), [(2025, 11), (2025, 12), (2026, 1)])


class ChartTests(SimpleTestCase):
    def setUp(self):
        self.s = stats.compute_stats([loan(), loan(status='active')], BOOKS, now=NOW)

    def test_every_chart_renders_html(self):
        for html in (
            charts.loans_per_month(self.s['month_series']),
            charts.loans_by_status(self.s['status_counts']),
            charts.top_books(self.s['top_books']),
            charts.loans_by_category(self.s['by_category']),
        ):
            self.assertIn('plotly-graph-div', html)

    def test_plotly_js_is_loaded_only_once(self):
        first = charts.loans_per_month(self.s['month_series'], first=True)
        other = charts.loans_per_month(self.s['month_series'])
        self.assertIn('cdn.plot.ly', first)
        self.assertNotIn('cdn.plot.ly', other)

    def test_chart_container_has_a_fixed_height(self):
        self.assertIn('<div style="height:320px">', charts.loans_per_month(self.s['month_series']))

    def test_status_axis_goes_at_least_to_100(self):
        html = charts.loans_by_status(self.s['status_counts'])
        self.assertIn('"range":[0,100]', html)

    def test_status_axis_grows_beyond_100(self):
        big = stats.compute_stats([loan() for _ in range(150)], BOOKS, now=NOW)
        self.assertNotIn('"range":[0,100]', charts.loans_by_status(big['status_counts']))

    def test_charts_survive_empty_data(self):
        empty = stats.compute_stats([], [], now=NOW)
        self.assertIn('plotly-graph-div', charts.top_books(empty['top_books']))
        self.assertIn('plotly-graph-div', charts.loans_by_category(empty['by_category']))


class DashboardViewTests(SessionTestCase):
    def test_student_is_redirected(self):
        self.login_as(STUDENT)
        self.assertRedirects(self.client.get('/dashboard/'), '/books/', fetch_redirect_response=False)

    def test_librarian_sees_kpis_and_four_charts(self):
        self.login_as(LIBRARIAN)
        data = stats.compute_stats([loan(), loan(status='active')], BOOKS, now=NOW)
        with mock.patch.object(stats, 'dashboard_stats', return_value=data):
            response = self.client.get('/dashboard/')
        self.assertEqual(response.status_code, 200)
        self.assertContains(response, 'plotly-graph-div', count=4)
        self.assertContains(response, 'Préstamos totales')
```

Completamos también `core/tests.py` con las pruebas de **control de acceso** de todo el sitio: un anónimo va al login, un estudiante no entra a páginas de bibliotecario, y viceversa. Primero agrega la importación:

MODIFICA `core/tests.py`

BUSCA este texto (está tal cual en el archivo):

```
from .pagination import paginate
```

y REEMPLÁZALO por:

```
from .pagination import paginate
from .testing import LIBRARIAN, STUDENT, SessionTestCase
```

Y luego agrega la clase al **final** del archivo:

core/tests.py — AGREGA AL FINAL

```
class AccessControlTests(SessionTestCase):
    def test_anonymous_is_sent_to_login_with_next(self):
        response = self.client.get('/books/')
        self.assertRedirects(response, '/login/?next=/books/', fetch_redirect_response=False)

    def test_student_cannot_open_librarian_pages(self):
        self.login_as(STUDENT)
        for url in ('/dashboard/', '/loans/', '/books/new/'):
            self.assertRedirects(self.client.get(url), '/books/', fetch_redirect_response=False)

    def test_librarian_cannot_open_student_pages(self):
        self.login_as(LIBRARIAN)
        self.assertRedirects(self.client.get('/loans/mine/'), '/books/', fetch_redirect_response=False)

    def test_home_redirects_by_role(self):
        self.login_as(LIBRARIAN)
        self.assertRedirects(self.client.get('/'), '/dashboard/', fetch_redirect_response=False)
        self.login_as(STUDENT)
        self.assertRedirects(self.client.get('/'), '/books/', fetch_redirect_response=False)
```

Ejecuta el conjunto completo:

TERMINAL

```
python manage.py test
```

SALIDA ESPERADA

```
Found 71 test(s).
System check identified no issues (0 silenced).
Creating test database for alias 'default'...
.....................................................................
----------------------------------------------------------------------
Ran 71 tests in 0.648s

OK
Destroying test database for alias 'default'...
```

✔ **Verificación final**

- `python manage.py test` termina con **OK** (71 pruebas).
- `/` lleva al bibliotecario al dashboard y al estudiante al catálogo.
- A 360, 768 y 1024 px no hay scroll horizontal en ninguna página.
- Los estudiantes no pueden abrir páginas de bibliotecario, ni al revés.

4.8 Errores comunes de la clase 4

Mensaje o síntoma	Causa probable	Solución
Las gráficas aparecen vacías	Sin internet: `plotly.js` se carga desde un CDN	Revisa la conexión; la primera gráfica debe cargar el script
Las gráficas se montan unas sobre otras	Falta el `<div style="height:...">` que envuelve cada gráfica	Revisa `_html()` en `charts.py`
Una gráfica se sale de su tarjeta o se corta	Plotly midió el contenedor antes de que se acomodara la página	Verifica el `post_script` en `_html()`
`ModuleNotFoundError: No module named 'plotly'`	Faltó instalar dependencias	`pip install -r requirements.txt`
El sitio no se ve igual en el celular	El navegador guardó el CSS viejo	Recarga forzada (**Ctrl + F5**)
Hay scroll horizontal a 360 px	Un elemento tiene ancho fijo	Abre F12, busca el elemento que se sale y revisa sus estilos
Las tablas no se vuelven tarjetas	Faltan las clases `table-cards` o los `data-label` en la plantilla	Compara con `books/list.html` y `loans/_table.html`
`Ran 70 tests` o menos	Falta pegar alguna prueba	Compara archivo por archivo con las clases anteriores

4.9 ¡Terminaste!

Revisa lo que construiste:

- Autenticación con Firebase (correo y contraseña) y **roles**.
- CRUD de libros con búsqueda, filtro y **paginación**.
- Préstamos con **reglas de negocio** y **transacciones** que protegen el stock.
- Un **seeder** que genera datos de prueba repetibles.
- Un **dashboard** con gráficas hechas en Python.
- Un sitio **responsive** a 360, 768 y 1024 px.
- **71 pruebas automáticas** que no tocan la base de datos real.
- Una arquitectura **por features** donde cada funcionalidad vive en su carpeta.

Retos para seguir aprendiendo

1. **Multa por atraso:** calcula días de retraso y muestra el valor en *Mis préstamos*.
2. **Renovar préstamo:** un botón que extienda 7 días la fecha límite, máximo una vez.
3. **Reservas:** si un libro no tiene copias, permitir "reservarlo" y avisar cuando se devuelva.
4. **Exportar a CSV:** un botón en el dashboard que descargue los préstamos filtrados.
5. **Portadas:** guardar una URL de imagen por libro y mostrarla en el catálogo.
6. **Despliegue:** publicar la app en un servicio como Render o Railway (necesitarás `DEBUG = False`, un `SECRET_KEY` propio en variables de entorno y la llave de Firebase como secreto del servicio).

Apéndices

A · Comandos de referencia

Qué quieres hacer	Comando
Activar el entorno virtual (Linux / Mac)	`source .venv/bin/activate`
Activar el entorno virtual (Windows PowerShell)	`.venv\Scripts\Activate.ps1`
Instalar las dependencias	`pip install -r requirements.txt`
Encender el servidor	`python manage.py runserver`
Apagar el servidor	`Ctrl + C`
Crear las tablas de sesiones	`python manage.py migrate`
Crear un usuario en Firebase	`python manage.py create_user correo@ejemplo.com "Nombre" --role librarian`
Sembrar libros y préstamos de prueba	`python manage.py seed`
Correr **todas** las pruebas	`python manage.py test`
Correr las pruebas de una feature	`python manage.py test features.loans`
Revisar que el proyecto no tenga errores de configuración	`python manage.py check`

B · Glosario

Término	Significado
App (de Django)	Un paquete de Python con una parte del proyecto. En esta guía, cada *feature* es una app
Atómico	Que ocurre completo o no ocurre; nunca queda a medias
Colección / documento	En Firestore, la "carpeta" y la "ficha" donde se guardan los datos
Decorador	Una función que envuelve a otra para agregarle comportamiento (por ejemplo, exigir login)
Entorno virtual	Carpeta (`.venv`) con las librerías de *este* proyecto, aisladas de las de otros
Feature	Una funcionalidad completa (libros, préstamos…) con todos sus archivos en una carpeta
Función pura	Función que solo recibe datos y devuelve un resultado, sin tocar bases de datos ni archivos
Middleware	Código que Django ejecuta en cada petición antes y después de la vista

Término	Significado
Migración	Instrucción que crea o cambia las tablas de la base de datos local
Mock	Un objeto falso que reemplaza temporalmente a uno real durante una prueba
Plantilla (template)	Archivo HTML con huecos que Django rellena con datos
Query string	La parte de la dirección después de `?`, por ejemplo `?q=python&page=2`
Responsive	Que el diseño se adapta al tamaño de la pantalla
Seeder	Script que crea datos de ejemplo
Servicio	En esta guía, el archivo `services.py` de cada feature: lo único que habla con Firestore
Sesión	Memoria del servidor sobre quién inició sesión en cada navegador
Transacción	Grupo de lecturas y escrituras que se ejecutan de forma atómica
Vista (view)	Función de Python que recibe una petición y devuelve una respuesta

C · Lista de entrega

Antes de dar tu proyecto por terminado, confirma:

- [ ] `python manage.py test` termina en **OK** con 71 pruebas.
- [ ] `.env` y `firebase/` **no** están en tu repositorio.
- [ ] Puedes registrarte, ingresar y salir.
- [ ] Un bibliotecario crea, edita y elimina libros.
- [ ] Un estudiante presta, devuelve y cancela préstamos, y las reglas lo frenan cuando corresponde.
- [ ] El dashboard muestra los 5 indicadores y las 4 gráficas.
- [ ] A 360, 768 y 1024 px no hay desplazamiento horizontal.
- [ ] Tu `requirements.txt` tiene las 5 librerías de la clase 1.